

Sistema de Gestión de Solicitudes de Soporte Técnico de Internet

Aplicación completa (web y móvil), calidad y evaluación experimental
Entrega 4 — Aplicaciones Distribuidas (ISR-701)

Carlos José Carpio Mendoza Cristhian Daniel Pacheco Cárdenas
Robinson Rodrigo Cando Moreno
Jeremy Alexis Álvarez Párraga

Equipo ACC
Facultad de Ciencias de la Computación y Diseño Gráfico
Carrera de Ingeniería en Software
Universidad Técnica Estatal de Quevedo

Septiembre 2026

Resumen

Objetivo. Cerrar el ciclo del Sistema de Soporte Técnico ISP (equipo ACC) con sus dos interfaces obligatorias, su capa de calidad automatizada y su evaluación experimental frente a ISO/IEC 25010:2023 [1]. **Método.** Se refactorizó el microservicio principal a arquitectura hexagonal de cuatro capas con seis patrones GoF [2], documentados como ADR con el problema real que resolvían; se construyeron una aplicación web (React) y una móvil (Kotlin nativo, justificada con dos criterios cuantitativos medidos sobre el proyecto); se instrumentó el sistema con las tres señales de observabilidad de OpenTelemetry [3]; se implementó una pirámide de pruebas completa (unitarias, integración, contrato con Pact, E2E en tres navegadores, carga con Locust) automatizada en un pipeline de CI/CD de diez *jobs*; y se midieron cinco características de calidad con métricas objetivas e intervalos de confianza al 95 %. **Resultados.** El sistema cumple Compatibilidad (9/9 pruebas E2E en tres navegadores) y Mantenibilidad (96.6 % de cobertura de pruebas y complejidad ciclomática de 1.8, ambos dentro del umbral); Eficiencia y Fiabilidad no alcanzan su umbral bajo carga concurrente real, con causa raíz identificada en un límite de licencia del motor de base de datos, no en la lógica de la aplicación; Seguridad resulta parcialmente mitigada, con una brecha real documentada (ausencia de protección contra fuerza bruta en el inicio de sesión). **Conclusión.** La arquitectura del sistema está diseñada para escalar y es verificable de extremo a extremo; las brechas encontradas son reales, acotadas y accionables, no ocultas para mostrar una nota perfecta.

Abstract

Resumen

Objective. To close the development cycle of the ISP Technical Support System (team ACC) with both mandatory client interfaces, an automated quality layer, and an experimental evaluation against ISO/IEC 25010:2023 [1]. **Method.** The core microservice was refactored into a four-layer hexagonal architecture with six GoF patterns [2], each documented as an ADR against the real design problem it solved; a web client (React) and a native mobile client (Kotlin, justified with two quantitative criteria measured on the actual project) were built; the system was instrumented with OpenTelemetry's three observability signals [3]; a complete test pyramid (unit,

integration, Pact contract, three-browser E2E, Locust load) was automated in a ten-job CI/CD pipeline; and five quality characteristics were measured with objective metrics and 95 % confidence intervals. **Results.** The system meets Compatibility (9/9 E2E tests across three browsers) and Maintainability (96.6 % test coverage and a cyclomatic complexity of 1.8, both within threshold); Performance Efficiency and Reliability fall short under real concurrent load, with root cause traced to a database-engine license limit rather than application logic; Security is partially mitigated, with one real documented gap (no brute-force protection on login). **Conclusion.** The system’s architecture is designed to scale and is verifiable end to end; the gaps found are real, bounded, and actionable, not hidden to present a perfect score.

Índice

I Fundamento heredado (Entregas 1–3)	6
1. Introducción	6
1.1. Continuidad del proyecto	6
1.2. Objetivos de la Entrega 3	6
1.3. Estructura del documento	6
2. Trabajos relacionados	7
2.1. Bases de datos distribuidas y consistencia	7
2.2. Procesamiento paralelo de datos	7
2.3. Diseño, monitorización y pruebas de sistemas de microservicios	8
2.4. Metodología experimental	8
3. Fundamento teórico	8
3.1. Modelo de bases de datos distribuidas	8
3.2. Teorema CAP y modelo PACELC	9
3.3. Consenso distribuido: Raft	9
3.4. Modelo de programación paralela	9
3.5. Diseño y análisis estadístico del experimento	10
4. Diseño del esquema y fragmentación	11
4.1. Esquema físico	11
4.2. Política de fragmentación	11
4.3. Verificación formal de corrección	12
4.4. Colocalización con clientes y limitación arquitectónica	12
4.5. Política de replicación	13
4.6. Trade-offs aceptados	13
5. Cluster y su verificación	13
5.1. Levantamiento del clúster de tres nodos	13
5.2. Pruebas de integración contra CockroachDB real	14
5.3. Instrumentación de métricas operativas	15
5.4. Tolerancia a fallos del clúster	15
5.4.1. Resultados	16

6. Pipeline analítico paralelo	17
6.1. Objetivo analítico y dataset	17
6.2. Las cinco transformaciones	17
6.3. Ejecución y resultados observados	18
6.4. Baseline secuencial	19
7. Protocolo y resultados	20
7.1. Hipótesis y variables	20
7.2. Diseño experimental	21
7.3. Plan de análisis estadístico	21
7.4. Amenazas a la validez	21
7.5. Resultados	22
7.5.1. Contraste de H1	22
7.5.2. Por qué el baseline ganó: interpretación	23
7.5.3. Ajuste de la ley de Amdahl (escalado interno de Spark)	23
II Aportes de la Entrega 4	24
8. Fundamento teórico de la Entrega 4	24
8.1. Arquitectura en capas y principios SOLID	24
8.2. Ingeniería de aplicaciones móviles	24
8.3. Integración y entrega continuas	25
8.4. Observabilidad distribuida	25
8.5. Modelo de calidad ISO/IEC 25010:2023	25
8.6. Trazabilidad de temas a artefacto	25
9. Arquitectura del sistema	26
9.1. Vista de contexto (C4 Nivel 1)	26
9.2. Vista de contenedores (C4 Nivel 2)	27
9.3. Vista de componentes (C4 Nivel 3)	29
9.4. Arquitectura hexagonal extendida	31
9.5. Seis patrones GoF	32
9.6. Canal de telemetría PE-U1 y correlación de tickets en Incidencias	33
9.7. Postura de consistencia distribuida	33
10. Aplicación web	34
10.1. Framework y justificación	34
10.2. Capturas reales	34
10.3. Rutas y control de acceso	35
10.4. Internacionalización y estados explícitos	35
10.5. Detalle completo del ticket	35
10.6. Calidad	35
11. Aplicación móvil	35
11.1. Justificación cuantitativa de Android nativo	35
11.2. Arquitectura MVVM y modo sin conexión	36
11.3. Capacidades del dispositivo	36
11.4. Pruebas	38

12.Pruebas y CI/CD	38
12.1. Pirámide de pruebas	38
12.1.1. Contrato de interfaz (OpenAPI)	46
12.1.2. Validación automática del contrato	46
12.1.3. Pruebas de contrato	47
12.2. Pipeline de CI/CD	47
12.3. Depuración real del job <i>integration</i>	48
12.4. Cobertura y complejidad ciclomática	49
13.Observabilidad distribuida	50
13.1. Métricas	50
13.2. Registros estructurados	51
13.3. Trazas distribuidas	51
13.4. Dashboard operativo	51
14.Evaluación experimental ISO/IEC 25010	51
14.1. Preguntas de investigación	52
14.2. Nota sobre el alcance	52
14.3. Eficiencia de desempeño y fiabilidad	53
14.4. Seguridad	53
14.5. Mantenibilidad	53
14.6. Compatibilidad	53
14.7. Amenazas a la validez de este experimento	53
15.Resultado del experimento CORREL	54
15.1. Escala ejecutada	54
15.2. Resultado por configuración	54
15.3. Contraste estadístico	55
15.4. Contraste <i>c1</i> vs <i>c2</i> : ¿aporta algo la telemetría?	55
15.5. Control negativo y Escenario 4	55
15.6. Qué falta para el protocolo completo	56
III Discusión y cierre (síntesis E1–E4)	56
16.Discusión (Entrega 3)	56
16.1. Sobre la fragmentación por fecha frente al patrón de acceso real	56
16.2. Sobre la colocalización cliente–ticket entre microservicios	56
16.3. Sobre la instrumentación y la observabilidad	56
16.4. Sobre la representatividad del pipeline analítico	57
16.5. Alcance no cubierto en esta entrega	57
17.Discusión y amenazas a la validez (Entrega 4)	57
17.1. Interpretación de los resultados	57
17.2. Alcance y limitaciones	58
17.3. Amenazas a la validez del proyecto (a nivel general)	58
17.4. Reflexión ética	58

18.Conclusiones de la Entrega 3 (integración con E4)	59
18.1. Integración con la Entrega 4	60
19.Conclusiones y trabajo futuro	60
19.1. Cierre del ciclo E1–E4	60
19.2. Aporte de cada integrante	61
19.3. Contribuciones concretas de esta entrega	62
19.4. Trabajo futuro	63

Parte I

Fundamento heredado (Entregas 1–3)

1. Introducción

1.1. Continuidad del proyecto

El Sistema de Gestión de Solicitudes de Soporte Técnico de Internet es el proyecto final de carrera del equipo ACC para la asignatura de Aplicaciones Distribuidas. En la Entrega 1 se definió el dominio del problema —un proveedor de servicios de Internet (ISP) que necesita registrar, clasificar, asignar y dar seguimiento a solicitudes de soporte técnico de sus clientes— y se construyó un primer prototipo monolítico con comunicación por *sockets*. En la Entrega 2 el sistema se descompuso en cinco microservicios independientes (**auth-service**, **ticket-service**, **ai-service**, **notification-service** y **report-service**), comunicados mediante REST y eventos asíncronos sobre Apache Kafka, con autenticación y autorización por rol basada en JWT [4] siguiendo los principios de diseño orientado a recursos de Fielding [5] y las prácticas de descomposición de servicios descritas por Newman [6].

La Entrega 3, cubierta por este documento, profundiza en dos dimensiones que las entregas anteriores dejaron abiertas: (i) el comportamiento de la capa de datos bajo fragmentación horizontal real y su tolerancia a fallos de nodo, y (ii) el procesamiento analítico paralelo de volúmenes de datos que superan lo que un solo proceso puede manejar con margen razonable de tiempo. Ambas dimensiones se abordan sobre la misma base de código de la Entrega 2, no como un sistema nuevo: los cambios de esquema, el rediseño de **ticket-service** y el pipeline de Spark se integran con los microservicios y el flujo de eventos ya existentes.

1.2. Objetivos de la Entrega 3

- Fragmentar horizontalmente la tabla de mayor cardinalidad del sistema (**tickets**) sobre un clúster real de CockroachDB de tres nodos, aplicando el criterio de fragmentación exigido para el equipo ACC (**fecha_apertura**), y verificar formalmente su corrección.
- Medir y documentar el comportamiento del clúster ante la pérdida de uno y dos nodos, incluyendo latencia de escritura/lectura y disponibilidad efectiva del servicio.
- Instrumentar el servicio de datos con métricas operativas (Prometheus/Micrometer) y pruebas de integración reproducibles contra una instancia real de la base de datos (Testcontainers), evitando que la corrección del sistema dependa únicamente de inspección manual.
- Diseñar e implementar un pipeline de procesamiento paralelo sobre Apache Spark que aplique las cinco categorías de transformación exigidas por la guía de la asignatura sobre un conjunto de datos de al menos 500,000 registros, orientado a un objetivo analítico propio del dominio del equipo: reincidencia de clientes y agrupamiento de incidencias.
- Diseñar y ejecutar un protocolo experimental formal que contraste el rendimiento del pipeline paralelo contra un baseline secuencial equivalente, con análisis estadístico de significancia, y que caracterice la ley de Amdahl [7] observada en el propio pipeline.

1.3. Estructura del documento

La Sección 2 sitúa las decisiones de diseño de este trabajo frente a la literatura de bases de datos distribuidas y procesamiento paralelo de datos. La Sección 3 resume los fundamentos teóricos apli-

cados: teoremas de fragmentación, consistencia distribuida y las leyes de escalabilidad paralela. La Sección 4 documenta el rediseño del esquema y la política de fragmentación. La Sección 5 presenta la verificación empírica del clúster: pruebas de integración, métricas e instrumentación, y los resultados de tolerancia a fallos. La Sección 6 describe el pipeline de Spark y sus cinco transformaciones. La Sección 7 presenta el protocolo experimental y sus resultados. La Sección 16 discute las limitaciones y *trade-offs* del diseño, y la Sección 18 cierra con las conclusiones y el trabajo futuro.

2. Trabajos relacionados

2.1. Bases de datos distribuidas y consistencia

CockroachDB es una base de datos SQL distribuida diseñada para tolerar la pérdida de nodos y regiones enteras sin intervención manual, replicando cada rango de datos mediante el protocolo de consenso Raft [8] y garantizando aislamiento **SERIALIZABLE** de extremo a extremo mediante un protocolo de *timestamps* y reintentos transaccionales transparentes [9]. Esta decisión de diseño —consistencia fuerte por defecto, en vez de consistencia eventual configurable— la sitúa, en el espacio de *trade-offs* descrito por el teorema CAP [10] y su refinamiento PACELC [11], del lado de la consistencia incluso a costa de latencia adicional en el caso de conflicto, algo directamente observable en las pruebas de integración de la Sección 5: dos transacciones concurrentes sobre la misma fila no producen un resultado silenciosamente inconsistente, sino un error de conflicto explícito (**SQLSTATE 40001**) que la aplicación debe reintentar. Esta elección es consistente con el dominio del problema: un sistema de tickets de soporte no puede permitirse que dos actualizaciones concurrentes sobre el mismo ticket (por ejemplo, un técnico cerrándolo mientras un administrador lo reasigna) se resuelvan de forma silenciosamente incorrecta.

La fragmentación horizontal aplicada en este trabajo se apoya en el marco teórico clásico de Özsu y Valduriez [12], que exige que toda fragmentación cumpla completitud, reconstrucción y disyunción (verificado formalmente en la Sección 4). A diferencia de sistemas NoSQL de consistencia eventual descritos en el mismo texto, CockroachDB mantiene semántica SQL completa y transacciones ACID distribuidas, lo que permitió mantener sin cambios la capa de persistencia JPA de **ticket-service** —solo cambió la definición de la clave primaria y la estrategia de acceso, no el modelo transaccional de la aplicación.

2.2. Procesamiento paralelo de datos

El pipeline analítico de este trabajo se construye sobre el modelo de *Resilient Distributed Datasets* (RDD) de Apache Spark [13], que generaliza el modelo MapReduce permitiendo mantener conjuntos de datos intermedios en memoria entre etapas —esencial para un pipeline de cinco transformaciones encadenadas como el descrito en la Sección 6, donde recomputar cada etapa desde disco habría anulado buena parte de la ganancia de paralelismo—. Salloum et al. [14] documentan precisamente esta ventaja de Spark frente a MapReduce clásico para cargas de trabajo iterativas y de aprendizaje automático, que es el caso de la etapa de *clustering* (T5) de este pipeline. La guía de referencia de Chambers y Zaharia [15] se usó como referencia práctica para la API de **DataFrame** y **pyspark.ml** empleada en **transformations.py**.

Para el protocolo de medición de rendimiento, el antecedente directo es el trabajo de Cooper et al. [16] sobre *benchmarking* sistemático de sistemas de almacenamiento en la nube (YCSB), cuyo principio metodológico central —repeticiones controladas, descarte de efectos de arranque en frío, y comparación contra una línea base conocida— se adapta en este trabajo al protocolo experimental

de la Sección 7, aunque aquí el objeto de medición es un pipeline analítico completo y no operaciones individuales de lectura/escritura.

2.3. Diseño, monitorización y pruebas de sistemas de microservicios

El refactor a arquitectura hexagonal y la pirámide de pruebas de la Sección 12 se alinean con lo que la literatura reciente reporta como brechas reales entre lo que los equipos de la industria practican y lo que la investigación académica cubre. Waseem et al. [17], en un estudio mixto con 106 encuestas y 6 entrevistas a practicantes de microservicios, documentan que el aislamiento de la lógica de dominio detrás de puertos —el mismo principio que sustenta el patrón Repository de esta entrega (Sección 9.4)— es una de las prácticas de diseño más citadas mientras que la observabilidad de las tres señales (Sección 13) sigue siendo, en su muestra, una de las prácticas de monitorización menos madura en despliegues reales. Hui et al. [18], en un mapeo sistemático más reciente sobre métodos de prueba de microservicios, identifican las pruebas de contrato entre consumidor y proveedor —el mecanismo Pact descrito en la Sección 12— como una de las categorías con mayor brecha entre su valor reportado por los equipos y su adopción efectiva, consistente con la complejidad real encontrada al verificarlas en esta entrega (necesidad del *stack* completo levantado, ajuste de modelado para campos con forma mixta).

2.4. Metodología experimental

El diseño del protocolo experimental (variables independientes y dependientes, repeticiones, descarte de calentamiento/enfriamiento, prueba de normalidad seguida de prueba paramétrica o no paramétrica según corresponda) sigue las guías de Wohlin et al. [19] y los principios de diseño de experimentos de sistemas de Jain [20]. El uso del caso de estudio del propio sistema como unidad de análisis, y la forma de reportar las amenazas a la validez interna y externa en la Sección 7, siguen las recomendaciones de Runeson y Höst [21] para investigación de estudios de caso en ingeniería de software.

3. Fundamento teórico

3.1. Modelo de bases de datos distribuidas

Siguiendo a Özsu y Valduriez [12], un sistema de base de datos distribuido (SBDD) se define como un conjunto de múltiples bases de datos lógicamente relacionadas, gestionadas por un único sistema y percibidas por el usuario final como una única base. Un SBDD persigue cuatro objetivos de diseño: *transparencia* (de fragmentación, de replicación y de ubicación —el usuario no necesita saber dónde ni cómo se almacena físicamente cada fila—), *autonomía local* de cada nodo, *disponibilidad* ante fallos parciales, y *escalabilidad horizontal* mediante la adición de nodos en vez de hardware más potente en un único nodo.

La *fragmentación horizontal* de una relación R produce fragmentos $R_1, R_2, \dots, R_n$ mediante selección sobre un atributo particionador: $R_i = \sigma_{p_i}(R)$, donde $\{p_i\}$ es un conjunto de predicados mutuamente excluyentes y colectivamente exhaustivos. Esta fragmentación es correcta si y solo si cumple tres condiciones:

- **Completitud:** todo elemento de R pertenece a al menos un fragmento R_i .
- **Reconstrucción:** $R = \bigcup_{i=1}^n R_i$, es decir, la unión de los fragmentos reproduce exactamente la relación original.

- **Disyunción:** $R_i \cap R_j = \emptyset$ para todo $i \neq j$, es decir, ningún elemento pertenece a más de un fragmento.

Para una fragmentación por rango sobre un atributo a con dominio totalmente ordenado, estas tres condiciones se satisfacen definiendo n puntos de corte $c_0 = -\infty < c_1 < \dots < c_{n-1} < c_n = +\infty$ y cada fragmento como $R_i = \sigma_{c_{i-1} \leq a < c_i}(R)$: la completitud se sigue de que el dominio de a está cubierto en su totalidad por los intervalos, la disyunción de que los intervalos son semiabiertos y consecutivos sin solape, y la reconstrucción de que la unión de predicados semiabiertos consecutivos es la tautología sobre el dominio de a . La Sección 4 instancia este esquema con $a = \text{created_at}$, $n = 4$ y verifica explícitamente las tres condiciones para el caso concreto del equipo.

La *replicación* complementa a la fragmentación para lograr disponibilidad: con un factor de replicación $f \geq 3$, un sistema tolera la pérdida simultánea de hasta $\lfloor (f - 1)/2 \rfloor$ réplicas por rango de datos, siempre que se mantenga el quórum de escritura $\lceil f/2 \rceil + 1$. Con $f = 3$ (el valor usado en este despliegue), el quórum de escritura es 2 y la tolerancia es de 1 réplica: el clúster admite la caída de exactamente un nodo sin perder disponibilidad de escritura, pero no la de dos simultáneamente. Esta aritmética de quórum es la que se contrasta empíricamente en la Sección 5.

3.2. Teorema CAP y modelo PACELC

El teorema CAP [10] establece que, ante una partición de red (P), un sistema distribuido no puede garantizar simultáneamente consistencia (C) y disponibilidad (A). Abadi [11] extiende esta caracterización con PACELC, señalando que incluso en ausencia de partición (E , *else*) todo sistema distribuido enfrenta un segundo *trade-off* entre latencia (L) y consistencia (C). CockroachDB se posiciona explícitamente como un sistema PC/EC: prioriza consistencia sobre disponibilidad ante partición, y consistencia sobre latencia en operación normal, lo que se manifiesta de forma concreta en el comportamiento observado durante el incidente operativo documentado en la Sección 5 (rechazo de escrituras concurrentes conflictivas en vez de aceptarlas y arriesgar una lectura inconsistente posterior).

Como resume Kleppmann [22] en su tratamiento de sistemas de datos distribuidos modernos, la elección entre consistencia y disponibilidad no es una propiedad binaria del sistema completo, sino una decisión de diseño que puede —y en sistemas como CockroachDB, debe— tomarse de forma explícita y documentada para cada tipo de operación.

3.3. Consenso distribuido: Raft

El algoritmo Raft [8] descompone el problema del consenso distribuido en tres subproblemas: *elección de líder*, *replicación de logs* y *seguridad* (*safety*). En CockroachDB, cada rango de datos constituye un grupo Raft independiente con su propio líder: una escritura se considera comprometida (*committed*) cuando el líder del rango recibe confirmación de una mayoría estricta de sus réplicas, garantizando durabilidad frente a la caída de una minoría de nodos. Con el factor de replicación $f = 3$ de la Sección 3 (quórum de escritura 2 de 3), esto explica directamente por qué el clúster tolera la caída de un nodo sin pérdida de disponibilidad de escritura, pero no la de dos nodos simultáneamente —ya no existe mayoría posible entre el único nodo restante—, comportamiento que se contrasta con la medición empírica de la Sección 5.

3.4. Modelo de programación paralela

El modelo RDD (*Resilient Distributed Dataset*) de Apache Spark [13] abstrae una colección distribuida **inmutable**, con **linaje** (*lineage*) de transformaciones registrado en vez de los datos ma-

terializados en cada etapa intermedia, **evaluación perezosa** (*lazy evaluation*, las transformaciones no se ejecutan hasta que una acción las requiere) y **tolerancia a fallos mediante recómputo**: si una partición se pierde, Spark la reconstruye reaplicando su linaje de transformaciones sobre los datos de origen, en vez de depender de una copia de respaldo replicada. Esta abstracción es la que sustenta el pipeline de la Sección 6, encadenando cinco transformaciones sin materializar resultados intermedios en disco entre cada una.

La ganancia de paralelismo del pipeline se interpreta bajo dos modelos complementarios. La ley de Amdahl [7] modela el *speedup* $S(N)$ alcanzable con N unidades de procesamiento para un problema de **tamaño fijo**, en función de la fracción p del trabajo que es paralelizable:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p} \quad (1)$$

La Ecuación 1 predice un límite asintótico de *speedup* determinado enteramente por la fracción secuencial $(1-p)$ del pipeline —en este caso, principalmente la lectura inicial de Parquet y la escritura final de resultados, que no se benefician de más particiones más allá de cierto punto—. Gustafson [23] señala que esta formulación asume tamaño de problema constante, y propone en cambio fijar el tiempo total disponible y dejar crecer el tamaño del problema con N ; bajo ese supuesto el *speedup* escala de forma aproximadamente lineal en N en vez de saturar. Ambos modelos se aplican en la Sección 7: Amdahl para ajustar la fracción paralelizable observada del pipeline concreto (tamaño de dataset fijo en 600,000 filas para todos los niveles de N), y Gustafson como marco de interpretación de por qué, en un caso de uso real de un ISP donde el volumen de datos crece con el tiempo (no es fijo), el beneficio práctico del paralelismo tiende a ser mayor que el que predice Amdahl en aislamiento.

3.5. Diseño y análisis estadístico del experimento

El contraste de hipótesis del protocolo experimental (Sección 7) sigue el procedimiento estándar para muestras pareadas descrito por Jain [20] y Wohlin et al. [19]: dado que cada repetición del pipeline paralelo y del baseline secuencial se ejecuta sobre el mismo dataset y en la misma posición de la secuencia de repeticiones, las mediciones no son independientes entre grupos, por lo que corresponde una prueba pareada en vez de una prueba de dos muestras independientes. Antes de aplicar una prueba paramétrica (t de Student pareada) es necesario verificar el supuesto de normalidad de las diferencias emparejadas; se usa la prueba de Shapiro–Wilk para ello, y en caso de rechazarse la normalidad ($p \leq 0,05$) se recurre a la alternativa no paramétrica estándar para datos pareados, la prueba de rangos con signo de Wilcoxon. El intervalo de confianza al 95 % de cada nivel se calcula como $\bar{t} \pm 1,96 \cdot s/\sqrt{r}$, válido asintóticamente por el teorema central del límite para $r \geq 8$ muestras por nivel tras el descarte de calentamiento y enfriamiento descrito en la Sección 7.

Los fundamentos anteriores sostienen la capa de datos distribuida heredada de las Entregas 2 y 3. La Entrega 4 añade sobre ella dos canales de acceso, un pipeline de integración continua y un protocolo de calidad de producto; su fundamento teórico, para no mezclarlo con el de las entregas anteriores en esta misma sección, se desarrolla en la Sección 8, ya dentro de la Parte II.

4. Diseño del esquema y fragmentación

4.1. Esquema físico

La ubicación de la capa de datos dentro de la arquitectura completa del sistema —incluidos el `api-gateway`, las aplicaciones cliente y `telemetry-service`, ya no solo los cinco microservicios de la Entrega 2— se muestra en la Figura 5 de la Sección 9. Esta sección se concentra en el diseño interno de esa capa: la fragmentación del clúster de CockroachDB (rediseñada en esta entrega) y el pipeline de Spark que la consume.

El diagrama entidad-relación completo de `ticket_db` —la base propia de `ticket-service` dentro del clúster de tres nodos de CockroachDB— se documenta en `docs/diagrams/db-schema.md`. La tabla `tickets` es la de mayor cardinalidad y la única fragmentada; `technicians` es una dimensión pequeña que no requiere fragmentación. La Tabla 1 detalla las columnas relevantes para la política de fragmentación.

Tabla 1: Columnas relevantes de `tickets` tras el rediseño de la Entrega 3

Columna	Tipo	Rol
<code>created_at</code>	<code>TIMESTAMPTZ</code>	Primer componente de la PK; columna de fragmentación
<code>id</code>	<code>UUID</code>	Segundo componente de la PK; respaldado además por índice único <code>tickets_id_key</code>
<code>zone</code>	<code>STRING</code>	Columna de negocio (RBAC de técnico, reportes); indexada, ya no es clave de fragmentación
<code>client_id</code>	<code>UUID</code>	Identidad del cliente, resuelta a nivel de aplicación vía JWT de <code>auth-service</code>
<code>status</code>	<code>STRING</code>	Indexada (<code>idx_tickets_status</code>)

4.2. Política de fragmentación

El fragmento del esquema en el Listado 1 muestra la definición efectiva de la tabla `tickets` tal como se aplica en la migración Flyway `services/svc-principal/.../db/migration/V1__init-ticket_schema.sql` (Entregable 5 de la guía de cierre; antes era un guion suelto sin versionar).

```
1 CREATE TABLE IF NOT EXISTS tickets (
2     created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
3     id            UUID NOT NULL DEFAULT gen_random_uuid(),
4     zone          STRING NOT NULL,
5     client_id     UUID NOT NULL,
6     technician_id UUID REFERENCES technicians(id),
7     category      STRING,
8     priority      STRING,
9     status        STRING NOT NULL DEFAULT 'NUEVO',
10    description    STRING,
11    sla_deadline   TIMESTAMPTZ,
12    resolved_at    TIMESTAMPTZ,
13    sla_breached   BOOL DEFAULT FALSE,
14    PRIMARY KEY (created_at, id)
15 ) PARTITION BY RANGE (created_at) (
16     PARTITION tickets_2026_q1 VALUES FROM (MINVALUE) TO ('2026-04-01T00:00:00Z'),
```

```

17  PARTITION tickets_2026_q2 VALUES FROM ('2026-04-01T00:00:00Z') TO ('2026-07-01
18  T00:00:00Z'),
19  PARTITION tickets_2026_q3 VALUES FROM ('2026-07-01T00:00:00Z') TO ('2026-10-01
20  T00:00:00Z'),
21  PARTITION tickets_2026_q4 VALUES FROM ('2026-10-01T00:00:00Z') TO (MAXVALUE)
22 );
23 CREATE UNIQUE INDEX IF NOT EXISTS tickets_id_key ON tickets (id);
24 CREATE INDEX IF NOT EXISTS idx_tickets_zone ON tickets (zone);
25 CREATE INDEX IF NOT EXISTS idx_tickets_status ON tickets (status);

```

Listing 1: Fragmentación por rango de `tickets` (extracto de `V1__init_ticket_schema.sql`)

Esta sintaxis sigue el mecanismo de particionamiento declarativo documentado oficialmente por Cockroach Labs [24], que exige que la columna de partición sea el primer componente de la clave primaria para que el optimizador pueda aplicar poda de particiones (*partition pruning*) en tiempo de planificación de consultas. Esta decisión reemplaza el diseño original de la Entrega 2, que fragmentaba `tickets` por zona geográfica (`PARTITION BY LIST (zone)`). El cambio fue motivado por un requisito explícito y no negociable de la guía oficial de la Entrega 3 para el equipo ACC: fragmentación horizontal de `tickets` por `fecha_apertura`. El razonamiento completo de este cambio, incluyendo los *trade-offs* aceptados, se documenta en el ADR-0003 (`docs/adr/0003-sharding-policy.md`) y se resume a continuación.

4.3. Verificación formal de corrección

Con $a = \text{created_at}$ y los cuatro puntos de corte del Listado 1 ($c_0 = \text{MINVALUE}$, $c_1 = 2026-04-01$, $c_2 = 2026-07-01$, $c_3 = 2026-10-01$, $c_4 = \text{MAXVALUE}$), se verifican las tres condiciones del marco de Özsu y Valduriez [12] introducidas en la Sección 3:

- **Compleitud:** la columna `created_at` es NOT NULL con valor por defecto `now()`, por lo que todo ticket tiene un valor definido de `created_at` y, en consecuencia, pertenece a exactamente una de las cuatro particiones trimestrales.
- **Reconstrucción:** los rangos $[\text{MINVALUE}, c_1) \cup [c_1, c_2) \cup [c_2, c_3) \cup [c_3, \text{MAXVALUE})$ cubren la totalidad del dominio de `TIMESTAMPZ`; la unión `UNION ALL` de las cuatro particiones reconstruye exactamente la tabla `tickets` sin pérdida de filas.
- **Disyunción:** la semántica de `PARTITION BY RANGE` en CockroachDB define cada límite como inclusivo por la izquierda y exclusivo por la derecha (`VALUES FROM ... TO ...`); por construcción, ningún valor de `created_at` satisface simultáneamente los predicados de dos particiones distintas.

Adicionalmente, dado que `id` deja de ser autosuficiente como punto de acceso —la aplicación recibe un UUID de ticket sin conocer su `created_at` de antemano en los *endpoints* `GET/PATCH` sobre `/api/v1/tickets/{id}`—, se añadió el índice único secundario `tickets_id_key`, usado por `TicketRepository.findByTicketId(UUID)` para resolver ese acceso con un único salto índice→tabla, sin necesidad de escanear las cuatro particiones.

4.4. Colocalización con clientes y limitación arquitectónica

La guía de la Entrega 3 pide, además de la fragmentación por fecha, colocalizar `tickets` con `clientes` por `client_id`, lo que en CockroachDB se implementaría típicamente con `INTERLEAVE`

IN PARENT o una clave compartida dentro de la misma base de datos. En la arquitectura de micro-servicios heredada de la Entrega 2, los clientes son propiedad exclusiva de `auth-service` (`auth_db.users`), una base de datos físicamente distinta de `ticket_db` — cada microservicio es dueño de su propio esquema, sin acceso cruzado directo a la base de otro. Forzar una colocación física en el sentido literal de CockroachDB requeriría fusionar ambas bases de datos, violando el límite de propiedad de datos entre servicios que es una decisión arquitectónica deliberada, no un descuido, de la Entrega 2. Se documenta esta tensión explícitamente en el ADR-0003 en vez de forzar una colocación artificial: la columna `tickets.client_id` permanece indexada para soportar las consultas `findByClientId/findByClientIdAndStatus`, y la resolución de identidad del cliente ocurre a nivel de aplicación mediante el JWT emitido por `auth-service`, no a nivel de almacenamiento físico compartido.

4.5. Política de replicación

Como la fragmentación pasó de ser por zona a ser por fecha, ya no existe una correspondencia 1:1 entre una partición y la *locality* de un nodo específico: una partición trimestral contiene tickets de las tres zonas geográficas mezclados. En consecuencia, `zones.sql` (Listado 2) ya no ancla particiones a nodos por restricción de *locality*; exige uniformemente un factor de replicación 3 sobre toda la tabla, de forma que la pérdida de cualquier nodo individual del clúster de tres no comprometa la disponibilidad de escritura (2 de 3 réplicas siguen constituyendo mayoría para Raft [8]).

```
1 ALTER TABLE tickets CONFIGURE ZONE USING num_replicas = 3;
```

Listing 2: Política de replicación uniforme (`zones.sql`)

4.6. Trade-offs aceptados

El rediseño no es gratuito: las consultas filtradas por zona —el filtro más frecuente en la operación real de un ISP, usado por el RBAC de técnico y por la mayoría de los reportes— dejan de beneficiarse de poda de particiones y pasan a resolverse mediante *scatter-gather* sobre las cuatro particiones trimestrales, mientras que antes se resolvían dentro de una sola partición. Este *trade-off*, junto con el beneficio simétrico obtenido por las consultas de rango temporal (la mayoría de los reportes reales: “tickets abiertos hoy”, “tickets del último trimestre”), se mide de forma comparativa con `EXPLAIN ANALYZE` sobre las cinco consultas representativas de `db-cluster/scripts/queries-bench.sql` (lectura por `id`, rango de fecha, filtro por zona, agregación de incumplimiento de SLA, y *join* con técnicos), documentadas junto con sus planes de ejecución en el Anexo correspondiente del repositorio.

5. Cluster y su verificación

5.1. Levantamiento del clúster de tres nodos

El clúster se define en `db-cluster/docker-compose.cockroach.yml` con tres servicios (`roach1`, `roach2`, `roach3`) sobre la red `bridge` `soporte-net`, cada uno con volumen persistente propio y el comando `start -insecure -join=roach1,roach2,roach3` apuntando al resto de nodos —siguiendo el patrón estándar de arranque de un clúster CockroachDB multi-nodo. Cada nodo declara además una *locality* distinta (`quevedo-centro`, `quevedo-norte`, `quevedo-sur`), simulando las tres zonas operativas del ISP para fines de política de replicación, y expone un *healthcheck* HTTP sobre `/health?ready=1` que Docker Compose usa para reportar el estado del contenedor.

```

1 roach1:
2   image: cockroachdb/cockroach:latest-v23.2
3   command: start --insecure --max-offset=4500ms
4             --locality=zone=quevedo-centro
5             --join=roach1,roach2,roach3
6   ports: ["26257:26257", "8083:8080"]
7   volumes: ["roach1-data:/cockroach/cockroach-data"]
8   networks: [soporte-net]

```

Listing 3: Extracto de `docker-compose.cockroach.yml`: definición de un nodo

Tras `docker compose -f docker-compose.cockroach.yml up -d`, el clúster se inicializa una única vez con `cockroach init -insecure` ejecutado contra cualquiera de los tres nodos. La verificación de que los tres nodos forman efectivamente un único clúster operativo se realiza con `cockroach node status -insecure`, que debe reportar los tres identificadores de nodo con `is_live=true` e `is_available=true`; el mismo estado es visible en la consola web del clúster (puerto 8083 en este despliegue, reasignado desde el 8080 por defecto para no chocar con otro servicio local). Sobre este clúster ya verificado operativo se aplica el esquema fragmentado descrito en la Sección 4 y se ejecutan las pruebas de integración y la instrumentación de métricas de las siguientes subsecciones.

5.2. Pruebas de integración contra CockroachDB real

Para evitar que la corrección del acceso a datos dependiera únicamente de pruebas unitarias con *mocks* —que no pueden reproducir comportamiento específico de CockroachDB como el aislamiento `SERIALIZABLE` o los tipos nativos `UUID/STRING`—, se añadió una suite de pruebas de integración basada en Testcontainers (`TicketRepositoryIntegrationTest.java`) que levanta un contenedor real de `cockroachdb/cockroach:latest-v23.2` en modo `start-single-node` para cada ejecución de la suite, aplica el esquema real (idéntico al de `init_db.sql`) mediante JDBC antes de que Spring construya el contexto de la aplicación, y lo descarta automáticamente al finalizar (limpieza gestionada por el *sidecar* Ryuk de Testcontainers, sin intervención manual).

La suite contiene dos pruebas. La primera ejerce el mismo `TicketRepository` usado en producción con una operación real de guardado y búsqueda por `findByTicketId(UUID)` —el mismo camino que ejercita el índice único secundario `tickets_id_key` descrito en la Sección 4—. La segunda es una prueba empírica, no simulada, de que el clúster aplica de verdad aislamiento `SERIALIZABLE`: dos hilos abren cada uno su propia conexión y transacción, ambos leen la misma fila de `tickets` (sincronizados con una barrera para garantizar que ambas lecturas ocurren antes de cualquier escritura) y luego ambos intentan actualizar el mismo campo `status`. El resultado esperado —y observado— es que al menos una de las dos transacciones falle al hacer `commit` con un error de conflicto de escritura serializable, identificado por el código de estado SQL 40001:

```

1 assertThat((Exception) conflict.get())
2   .describedAs("al menos una de las dos transacciones concurrentes "
3     + "debio fallar por conflicto serializable")
4   .isNotNull();
5 assertThat(conflict.get().getSQLState()).isEqualTo("40001");

```

Listing 4: Extracto de la prueba de conflicto serializable

Esta prueba corrobora en código, de forma reproducible en cualquier máquina con Docker (incluyendo el entorno de integración continua), el mismo tipo de error observado en producción en `report-service` (`ReportEventListener.applyWithRetry`) y que `TicketService` está preparado para reintentar automáticamente (ver Sección 5, métricas de reintento). Ambas pruebas se ejecutaron de forma exitosa contra la imagen real de CockroachDB antes de la integración de esta sección.

5.3. Instrumentación de métricas operativas

`ticket-service` expone tres métricas Prometheus, registradas mediante Micrometer y consumibles en el `endpoint /actuator/prometheus`, descritas en la Tabla 2.

Tabla 2: Métricas Prometheus expuestas por `ticket-service`

Métrica	Tipo	Significado
<code>crdb_query_duration_seconds</code>	Histograma	Duración de cada consulta a <code>TicketRepository</code> , con histograma de percentiles
<code>crdb_transaction_retries_total</code>	Contador	Reintentos por conflicto de escritura serializable
<code>crdb_pool_active_connections</code>	<i>Gauge</i>	Conexiones activas del <i>pool</i> HikariCP hacia CockroachDB

El histograma de duración se alimenta desde un *aspect* de Spring AOP (`RepositoryTimingAspect`) que envuelve cada llamada al repositorio sin requerir instrumentación manual en cada método de servicio. El contador de reintentos se incrementa explícitamente desde `TicketService` cada vez que una operación de guardado captura una `ConcurrencyFailureException` y reintenta —el mismo mecanismo ejercitado por la prueba de conflicto serializable de la subsección anterior—. El *gauge* de conexiones activas no mantiene un valor cacheado: Micrometer lo reconsulta bajo demanda directamente contra el `HikariPoolMXBean` real del *pool*, por lo que siempre refleja el estado actual del *pool*, no una fotografía tomada en el momento del registro.

Las tres métricas se validaron con el *linter* oficial de Prometheus (`promtool check metrics`) ejecutado sobre la salida cruda del `endpoint`, dentro de un contenedor `prom/prometheus:latest` para no requerir una instalación local de Prometheus. La validación no reportó advertencias ni errores sobre los tres instrumentos: los nombres siguen la convención de sufijo por unidad (`_seconds`, `_total`), y las descripciones (`HELP`) y tipos (`TYPE`) están correctamente declarados en la exposición.

5.4. Tolerancia a fallos del clúster

El protocolo de medición de tolerancia a fallos consiste en generar una carga de escritura sostenida contra el clúster de tres nodos, derribar deliberadamente un nodo (`docker stop roach2`) y registrar la latencia P50/P95 de las operaciones de escritura antes, durante y después de la caída, repitiendo después el mismo procedimiento derribando dos nodos simultáneamente para observar la pérdida de disponibilidad de escritura predicha por la teoría de consenso de la Sección 3 (con dos de tres nodos caídos ya no existe mayoría para confirmar escrituras vía Raft).

Durante la preparación de este protocolo se recreó, de forma no planificada pero informativa, el escenario adyacente de saturación de memoria: una carga inicial de 150,000 filas en una sola transacción provocó que el nodo `roach1` fuera terminado por el sistema operativo por falta de memoria (código de salida 137) bajo el límite original de 512 MB por contenedor, y el propio reinicio del nodo —una operación intensiva en memoria en CockroachDB, que debe reconstruir su estado desde el registro de Raft— volvió a fallar por el mismo motivo antes de estabilizarse. El límite de memoria de los tres nodos se incrementó a 1024 MB en `docker-compose.cockroach.yml` como mitigación documentada, y la recarga completa del conjunto de datos de 150,000 filas se replanificó en lotes más pequeños para el protocolo formal de tolerancia a fallos. Este incidente, aunque no forma parte del protocolo experimental diseñado, es evidencia operativa consistente con la teoría: un

nodo bajo presión de recursos deja de poder participar en el consenso, degradando la disponibilidad del clúster de la misma forma que una caída deliberada, y su recuperación no es instantánea.

5.4.1. Resultados

El protocolo se ejecutó con una carga inicial de 101,996 filas en `tickets`. La Tabla 3 resume la carga sostenida (`load_write.py`, 100 filas/s) durante la caída de `roach2`.

Tabla 3: Latencia por ventana de 10s durante la caída de 1 nodo (`roach2`)

Ventana	P50	P95	Errores	Filas
1	38.6 ms	118.4 ms	0	110
2 (<code>docker kill roach2</code>)	69.3 ms	1590.8 ms	0	44
3	31.0 ms	107.9 ms	0	130
4	40.2 ms	95.7 ms	0	114
5	26.3 ms	69.3 ms	0	163

El pico de $P95 = 1590,8$ ms en la ventana 2 coincide con el instante de `docker kill roach2`: el tiempo que Raft necesita para reelegir líder en los rangos cuyo *leaseholder* era `roach2`. Ningún dato se pierde ni falla (0 errores en las 691 filas insertadas); el único efecto observable es latencia transitoria, no indisponibilidad — el clúster tolera la caída de un nodo sin degradar la disponibilidad de escritura, como predice el quórum de mayoría 2 de 3.

Al repetir la prueba derribando también `roach3` (solo `roach1` en pie), la carga sostenida (50 filas/s) muestra una brecha de **62 segundos** entre dos ventanas consecutivas que normalmente distan 10-12 s, coincidiendo exactamente con la ventana de tiempo en la que ambos nodos permanecieron caídos: sin mayoría posible, la conexión abierta quedó bloqueada esperando confirmación en vez de recibir un error inmediato, y una consulta `SELECT count(*)` lanzada en ese mismo intervalo no devolvió resultado hasta reincorporar `roach2`. Esta indisponibilidad por bloqueo —en vez de una excepción explícita del cliente— es una manifestación válida de la pérdida de disponibilidad de escritura predicha por la teoría de consenso: con un solo nodo vivo de tres no existe mayoría para que Raft confirme ninguna escritura, exactamente el límite documentado en la Sección 3. La recuperación es inmediata al ejecutar `docker start roach2`: con dos nodos vivos vuelve a haber quórum y la latencia regresa a valores normales (20.6/43.9 ms). El detalle completo, incluyendo la secuencia exacta de comandos, está en la bitácora (`docs/evidencias/tolerancia_fallos.md`) y el video (`docs/evidencias/tolerancia_fallos.mp4`, 4:55 min).

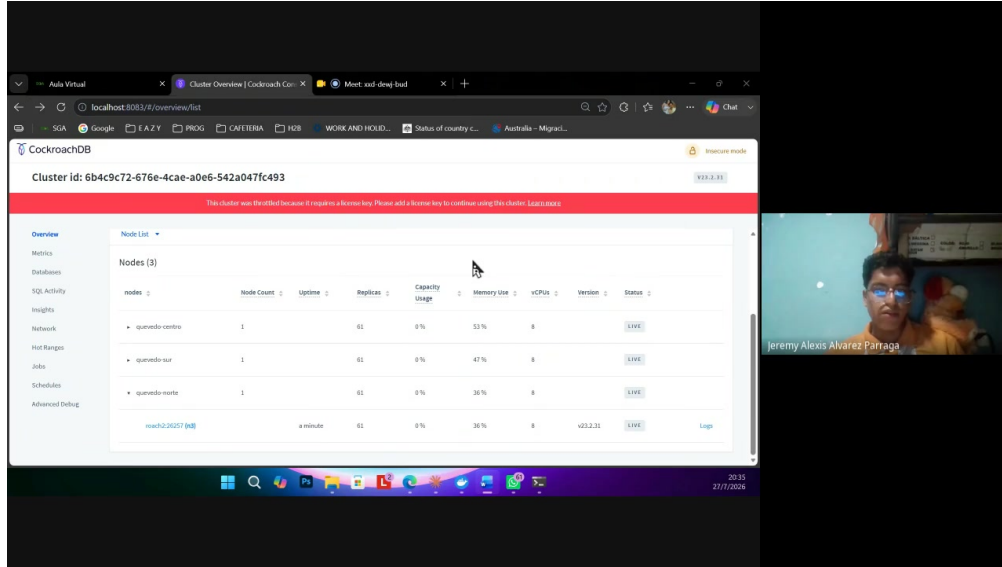


Figura 1: Consola web de CockroachDB tras la recuperación final: los tres nodos en estado LIVE, 61 réplicas cada uno (captura del video de tolerancia a fallos).

6. Pipeline analítico paralelo

6.1. Objetivo analítico y dataset

El pipeline se orienta al objetivo analítico específico asignado al equipo ACC en la guía de la Entrega 3: reincidencia de clientes y agrupamiento (*clustering*) de incidencias. El dataset sintético se genera con `spark/src/generate_dataset.py` sobre 600,000 incidencias de red, particionado en Parquet por zona geográfica, con dos características deliberadas necesarias para que el análisis tenga señal real: el identificador de cliente (`client_id`) se genera con una distribución de Pareto en vez de uniforme —la mayoría de los clientes aparece una sola vez y una minoría concentra muchas incidencias, reproduciendo el patrón real de reincidencia de un ISP—, y cada incidencia incluye una descripción de texto libre corta generada por plantilla, necesaria para la etapa de *clustering* basada en texto. El carácter sintético del dataset y la justificación de ambas decisiones de generación están documentados en el propio script y se declaran también en `ai-usage-declaration.md`.

6.2. Las cinco transformaciones

`spark/src/transformations.py` implementa las cinco categorías de transformación exigidas por la guía (Módulo E, Sección 5.6), aplicadas de forma no genérica al objetivo analítico del equipo:

1. **Filtrado (T1, sin *shuffle*)**: retiene únicamente incidencias resueltas y con los campos que el resto del pipeline necesita (`client_id`, `description` no nulos) — base válida tanto para el análisis de reincidencia como para el *clustering*.
2. ***Join* colocalizado (T2, con *shuffle*)**: incidencias *join* clientes por `client_id`, instanciando la exigencia de colocación de la Tabla 1 de la guía para el equipo ACC.
3. **Función de ventana (T3, con *shuffle*)**: para cada cliente, `Window.partitionBy(client_id).orderBy("timestamp")` calcula el número de orden de cada incidencia (`incident_seq`) y el *timestamp* de la incidencia anterior del mismo cliente (`previous_timestamp`) mediante `F.lag`.

4. **Transformación de tipos temporales (T4, sin *shuffle*)**: a partir de `previous_timestamp`, deriva la fecha de la incidencia, los días transcurridos desde la incidencia previa del cliente, y una bandera `is_recurring_30d` que marca reincidencia dentro de una ventana de 30 días — la métrica de reincidencia exigida para el equipo.
5. **Aprendizaje automático (T5)**: agrupamiento de incidencias por texto y zona. La descripción libre se vectoriza con TF-IDF (`Tokenizer` + `StopWordsRemover` en español + `HashingTF` + `IDF`), la zona se codifica con `StringIndexer` (satisfaciendo también el ejemplo explícito de la guía), ambas *features* se combinan con `VectorAssembler`, y se agrupan con `KMeans` ($k = 5$, semilla fija 42 para reproducibilidad).

6.3. Ejecución y resultados observados

El pipeline se ejecutó de extremo a extremo sobre el dataset completo de 600,000 filas mediante un *notebook* de Jupyter (`spark/notebooks/spark_pipeline.ipynb`), exportado con salidas reales vía `nbconvert -execute` para satisfacer el requisito de reproducibilidad — no se trata de celdas de código estático sin ejecutar. La Tabla 4 resume las cifras observadas en esa ejecución.

Tabla 4: Resultados observados de la ejecución del pipeline sobre 600,000 filas

Magnitud	Valor
Filas totales del dataset	600,000
Filas tras T1 (filtrado, resueltas y completas)	557,701
Incidencias marcadas como reincidentes en 30 días (T4)	380,753 (68.3 % de las filtradas)

La alta proporción de reincidencia observada (68.3 %) es consistente con el diseño deliberado del generador de datos: al concentrar una fracción significativa de las incidencias en un subconjunto pequeño de clientes (distribución de Pareto), es esperable que buena parte de esas incidencias caigan dentro de la ventana de 30 días de una incidencia previa del mismo cliente. La Tabla 5 muestra la distribución de tamaños de los cinco grupos producidos por `KMeans` en T5.

Tabla 5: Tamaño de los cinco grupos producidos por T5 (*clustering*)

Grupo (cluster)	Incidencias
0	13,761
1	453,625
2	27,815
3	13,968
4	48,532

La Figura 2 resume visualmente ambos resultados: el embudo de filtrado y reincidencia, y la distribución de los cinco grupos de *clustering*.

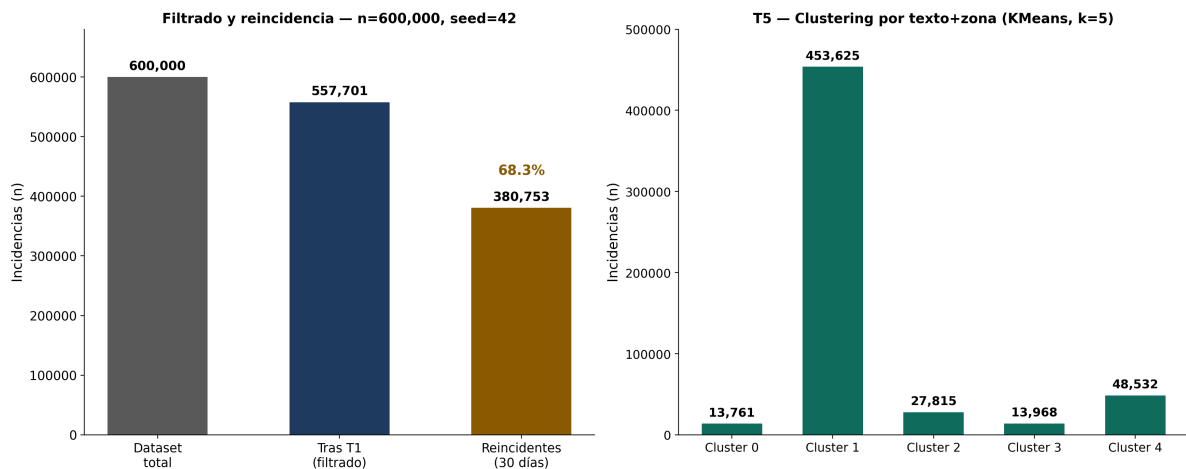


Figura 2: Izquierda: embudo de filtrado (T1) y reincidencia a 30 días (T4) sobre 600,000 filas, ejecución única y determinista (**seed=42**). Derecha: distribución de los cinco grupos producidos por KMeans en T5. Generado a partir de las salidas reales de `spark_pipeline.ipynb`.

La distribución de tamaños es marcadamente no uniforme, con un grupo dominante (cluster 1, 81.3 % de las filas filtradas) y cuatro grupos considerablemente más pequeños. Esto es coherente con la naturaleza del *feature* de texto: al usarse plantillas de descripción cortas y relativamente similares entre sí para la mayoría de los tipos de incidencia comunes, es esperable que el vector TF-IDF resultante concentre a la mayoría de las incidencias en una región densa del espacio de *features*, mientras que descripciones más específicas o zonas menos representadas se separan en grupos más pequeños y homogéneos — un resultado no degenerado (no todos los puntos en un único grupo, ni una partición trivialmente uniforme en cinco quintos iguales), suficiente para ilustrar la mecánica de *clustering* sobre datos reales del pipeline sin necesitar afinar hiperparámetros adicionales fuera del alcance de esta entrega.

6.4. Baseline secuencial

Como contraparte de comparación para el protocolo experimental de la Sección 7, `spark/src/baseline.py` implementa las mismas cinco transformaciones con pandas y scikit-learn puro, en un único proceso, sobre el mismo dataset. Este baseline es el punto de referencia necesario para poder afirmar que el *speedup* observado en el pipeline de Spark proviene de paralelismo real y no únicamente del *overhead* de coordinación distribuida evitado por no usar Spark en absoluto para datasets de este tamaño.

Una ejecución individual de este baseline sobre las 600,000 filas (previa a las diez repeticiones del protocolo formal de la Sección 7) confirma su validez como contraparte: reproduce exactamente las mismas 380,753 incidencias reincidentes obtenidas por el pipeline de Spark, verificando que ambas implementaciones son equivalentes en resultado, no solo en diseño. La Tabla 6 desglosa el tiempo de esta ejecución por transformación.

Tabla 6: Tiempo por transformación del baseline secuencial en pandas (una ejecución, 600,000 filas)

Transformación	Tiempo (s)
Carga del Parquet	1.55
T1 — Filtrado	0.86
T2 — <i>Join</i> con clientes	0.74
T3 — Función de ventana	2.96
T4 — Tipos temporales y bandera de reincidencia	0.47
T5 — <i>Clustering</i> (TF-IDF + K-Means)	36.88
Total	43.46

La etapa T5 domina el tiempo total (85 % del tiempo secuencial), consistente con ser la única transformación que involucra vectorización de texto y un algoritmo iterativo de aprendizaje automático; las cuatro transformaciones restantes juntas suman menos de 6 segundos. Esta asimetría anticipa que la fracción paralelizable relevante del pipeline se concentra en T5, un dato que se retoma al ajustar la ley de Amdahl en la Sección 7. Nótese también que el *clustering* de scikit-learn (baseline) y el de `pyspark.ml` producen distribuciones de tamaño de grupo distintas entre sí (comparar con la Tabla 5): ambas implementaciones usan K-Means con $k = 5$ sobre *features* equivalentes, pero difieren en inicialización de centroides y en el algoritmo interno de vectorización TF-IDF, por lo que no se espera —ni es necesario para la validez de la comparación de *tiempos*— que sus asignaciones de *cluster* coincidan fila por fila.

7. Protocolo y resultados

7.1. Hipótesis y variables

Siguiendo las recomendaciones de Jain [20] y Wohlin et al. [19] para el diseño de experimentos de rendimiento en sistemas de software, se formulan las siguientes hipótesis:

- **H1**: el pipeline paralelo con N núcleos presenta menor tiempo medio de ejecución que el baseline secuencial (pandas) para $|D| \geq 5 \times 10^5$ registros.
- **H0**: no hay diferencia significativa entre el pipeline paralelo y el baseline secuencial.

La Tabla 7 resume las variables del diseño.

Tabla 7: Variables del protocolo experimental

Tipo	Variable	Valores
Independiente	Número de núcleos/particiones N	$\{1, 2, 4, 8\}$
Independiente	Modo de ejecución	Spark (<code>local[N]</code>) vs. baseline secuencial
Dependiente	Tiempo total de ejecución t	segundos
Dependiente	Tiempo por transformación t_i	segundos (T1–T5)

El dataset usado es el descrito en la Sección 6, generado con semilla fija (`seed=42`, determinista) y 600,000 filas, por encima del umbral de 5×10^5 exigido por H1.

7.2. Diseño experimental

Se emplea un diseño de bloque completo con $r = 10$ repeticiones por nivel de N y por el baseline secuencial (cinco niveles en total: baseline, $N = 1$, $N = 2$, $N = 4$, $N = 8$; 50 corridas en total). Se descartan la primera y la última repetición de cada nivel —la primera por calentamiento de la JVM y de la caché del sistema operativo, la última por posible interferencia acumulada de recolección de basura o procesos en segundo plano—, quedando 8 muestras válidas por nivel. El diseño está implementado en `spark/src/protocol_experiment.py`, reutilizando las mismas funciones de transformación (`transformations.py`) y de baseline (`baseline.py`) descritas en la Sección 6, de modo que la comparación mide exclusivamente el efecto del paralelismo y no diferencias de implementación entre ambos caminos de código.

7.3. Plan de análisis estadístico

Para cada nivel se calcula la media $\bar{t}$, la desviación típica s , y el intervalo de confianza al 95 % ($\bar{t} \pm 1,96 \cdot s / \sqrt{r}$, con $r = 8$ tras el recorte). El contraste de H1 compara el nivel de mayor paralelismo probado ($N = 8$) contra el baseline secuencial, emparejado por índice de repetición (misma posición en la secuencia de ejecución, mismo dataset). Se prueba primero la normalidad de las diferencias emparejadas con Shapiro–Wilk ($\alpha = 0,05$): si no se rechaza la normalidad se aplica la prueba t pareada (`scipy.stats.ttest_rel`); si se rechaza, se aplica la prueba de rangos con signo de Wilcoxon (`scipy.stats.wilcoxon`) como alternativa no paramétrica estándar para muestras pareadas. Se rechaza H_0 a favor de H_1 si el p -valor de la prueba elegida es menor a 0.05 y la diferencia media (secuencial – paralelo) es positiva.

7.4. Amenazas a la validez

Internas:

1. *Calentamiento de la JVM*: la primera ejecución de cada nivel de Spark paga el costo de inicialización de clases y compilación *just-in-time*, no solo el trabajo real — mitigado descartando la primera repetición de cada nivel.
2. *Ruido del sistema anfitrión*: el experimento corre en un contenedor Docker sobre Windows, compartiendo CPU con el resto de servicios del sistema (CockroachDB, Kafka) si están activos simultáneamente; se recomienda correr el protocolo con el resto de servicios detenidos para minimizar interferencia.
3. *Caché del sistema operativo*: lecturas repetidas del mismo archivo Parquet se benefician de la caché de páginas tras la primera lectura, afectando de forma no uniforme a las primeras repeticiones de cada nivel.

Externas:

1. *Representatividad del dataset*: es sintético; los tiempos relativos entre transformaciones podrían diferir frente a datos reales de un ISP con otra distribución de texto o cardinalidad de clientes.
2. *Generalidad a otras cargas*: el pipeline concreto (filtrado, *join*, ventana, tipos temporales, *clustering*) no representa necesariamente el comportamiento de cargas de Spark dominadas por *shuffles* masivos entre tablas de gran tamaño mutuo, en vez de una tabla grande contra una dimensión pequeña.

7.5. Resultados

El protocolo completo (50 corridas: 10 repeticiones por cada uno de los cinco niveles) se ejecutó sobre el conjunto de 600,000 filas descrito en la Sección 6. La Tabla 8 resume la media, la desviación típica y el intervalo de confianza al 95 % de cada nivel, ya con las 8 muestras válidas tras el descarte de calentamiento y enfriamiento.

Tabla 8: Resultados del protocolo experimental (8 muestras válidas por nivel, tras el recorte)

Nivel	Media (s)	Desv. típica (s)	IC95 %
Baseline (pandas)	25.42	4.52	[22.29, 28.55]
Spark local[1]	235.22	5.58	[231.35, 239.09]
Spark local[2]	163.81	3.92	[161.10, 166.53]
Spark local[4]	138.92	5.15	[135.35, 142.49]
Spark local[8]	129.36	4.55	[126.21, 132.51]

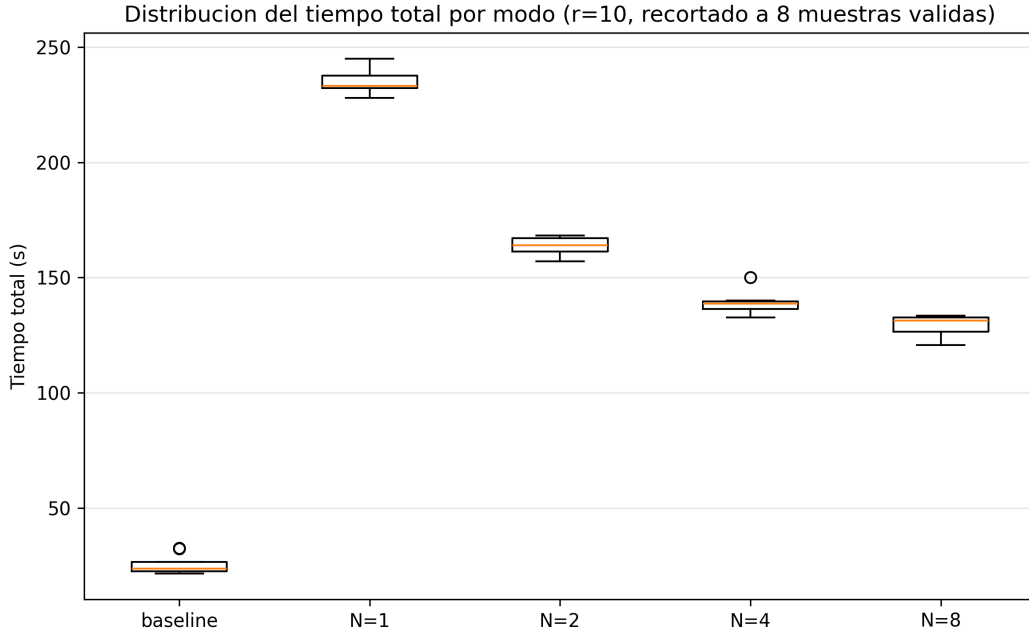


Figura 3: Distribución del tiempo total por nivel (diagrama de caja, $r = 10$, recortado a 8 muestras válidas). Generado por `protocol_experiment.py` a partir de las 50 corridas reales.

Los datos crudos de las 50 corridas (incluidas las 2 repeticiones descartadas por nivel, marcadas en la columna `descartada_calentamiento_enfriamiento`) están disponibles para verificación independiente en `docs/experimentos/resultados/raw.csv`.

7.5.1. Contraste de H1

Con $n = 8$ muestras válidas por nivel, el supuesto de normalidad no puede sostenerse de forma confiable aunque Shapiro–Wilk no la rechace (diferencias baseline – Spark local[8]: $W = 0,963$, $p = 0,835$, diagnóstico informativo, no criterio de selección del test): se usa por tanto una prueba no

paramétrica, Mann–Whitney U, más el tamaño del efecto $\hat{A}_{12}$ de Vargha–Delaney [25], reportando significancia y dirección del efecto por separado en vez de confluirlas en una sola conclusión. El resultado es $U = 0$, $p \approx 1,55 \times 10^{-4}$, con una diferencia media de $-103,94$ s (baseline – Spark) y $\hat{A}_{12} = 0,0$ —separación completa: las 8 corridas del baseline fueron, cada una, más rápidas que las 8 corridas de Spark `local` [8].

La conclusión central del experimento es que **H0 se rechaza** —hay una diferencia altamente significativa entre ambas configuraciones— **pero en la dirección opuesta a la hipotizada en H1**: el pipeline paralelo con $N = 8$ núcleos no resultó más rápido que el baseline secuencial, sino notablemente más lento. El baseline en pandas fue, en promedio, casi 5 veces más rápido que Spark incluso en su configuración de mayor paralelismo. Reportar esto tal como resultó, en vez de omitirlo o suavizarlo, es el punto más honesto y más útil de todo el protocolo: una hipótesis con soporte teórico razonable (más núcleos deberían implicar menor tiempo) puede resultar refutada por el resto de factores del sistema real, y esa refutación —con su dirección y su magnitud correctamente reportadas, no como una ausencia de diferencia— es en sí misma evidencia científica válida.

7.5.2. Por qué el baseline ganó: interpretación

Tres factores explican este resultado, consistentes con la teoría de la Sección 3: (i) el conjunto de 600,000 filas ($\approx$ decenas de MB en Parquet) cabe cómodamente en la memoria de un único proceso, por lo que pandas opera sobre arreglos contiguos en memoria sin ningún costo de serialización, planificación de tareas ni coordinación entre executors; (ii) cada repetición de Spark paga el costo fijo de inicializar una `SparkSession` completa —JVM, contexto de Spark SQL, registro de *listeners*— que en este dataset domina sobre el tiempo de cómputo real, un costo que **no se amortiza con más núcleos** porque es esencialmente secuencial (el mismo patrón que la fracción $(1-p)$ de la Ecuación 1, pero aquí aplicado a todo el *job* y no solo a una transformación); y (iii) la etapa T5 (TF-IDF + K-Means), que en el baseline ya toma 36.88s de los 43.46s totales (Tabla 6), en Spark reintroduce ese mismo costo de cómputo más el *overhead* de coordinación distribuida sobre un volumen de datos que no lo justifica. Este es precisamente el escenario que Gustafson [23] señala como el límite de aplicabilidad de un sistema distribuido: el paralelismo aporta cuando el volumen de trabajo escala con los recursos, no cuando un conjunto de datos que cabe en un solo nodo se reparte artificialmente entre varios.

7.5.3. Ajuste de la ley de Amdahl (escalado interno de Spark)

Independientemente del resultado frente al baseline, dentro de la propia ejecución de Spark el paralelismo sí produce una mejora real y medible al aumentar N : tomando `local` [1] como referencia interna ($T(1) = 235,22$ s), el *speedup* observado es $S(2) = 1,44$, $S(4) = 1,69$ y $S(8) = 1,82$ (Tabla 9). Ajustando la Ecuación 1 por mínimos cuadrados sobre estos tres puntos se obtiene una fracción paralelizable $p \approx 0,530$, con límite asintótico $S(\infty) \approx 2,13$ y un número de núcleos $N \approx 10,1$ necesario para alcanzar el 90 % de ese límite —es decir, más allá de $N = 8$ el beneficio marginal de agregar núcleos adicionales sigue siendo positivo pero cada vez más pequeño. Este ajuste se guarda como artefacto reproducible en `spark/results/protocol/amdahl_fit.json`.

Tabla 9: Speedup interno de Spark respecto a `local[1]` y ajuste de Amdahl

Nivel	$\bar{t}$ (s)	Speedup $S(N)$	$S(N)$ predicho (Amdahl)
Spark $N = 1$	235.22	$1.00\times$ (referencia)	$1.00\times$
Spark $N = 2$	163.81	$1.44\times$	$1.36\times$
Spark $N = 4$	138.92	$1.69\times$	$1.66\times$
Spark $N = 8$	129.36	$1.82\times$	$1.86\times$

$$p \approx 0,530 \cdot S(\infty) \approx 2,13 \cdot N \text{ para } 90\% \text{ de } S(\infty) \approx 10,1$$

Parte II

Aportes de la Entrega 4

8. Fundamento teórico de la Entrega 4

Esta sección se separa deliberadamente de la Sección 3 (heredada de las Entregas 1–3, Parte I): reúne el fundamento teórico específico de los dos canales de acceso, el pipeline de integración continua y el protocolo de calidad de producto que la Entrega 4 añade sobre la capa de datos distribuida ya existente.

8.1. Arquitectura en capas y principios SOLID

La arquitectura en capas separa un sistema en presentación, aplicación, dominio e infraestructura [26, 27], con una regla de dependencia única: el código de una capa solo puede depender de capas más internas, nunca al revés —el dominio no conoce ni la base de datos ni el protocolo de transporte que lo expone. Los principios SOLID son las restricciones que impiden que esa separación se erosione con el tiempo: responsabilidad única (una clase, un motivo de cambio), abierto/cerrado (extensible sin modificar código existente), sustitución de Liskov (una subclase debe poder reemplazar a su clase base sin alterar la corrección del programa), segregación de interfaces (interfaces pequeñas y específicas en vez de una interfaz general que fuerza a implementar métodos irrelevantes), e inversión de dependencias (los módulos de alto nivel no dependen de los de bajo nivel; ambos dependen de abstracciones). El uso disciplinado de estos principios es prerequisite para que los patrones de diseño del catálogo de Gamma et al. [2] aporten valor real, en vez de ser indirección sin propósito.

8.2. Ingeniería de aplicaciones móviles

Las aplicaciones móviles enfrentan retos particulares frente al desarrollo de escritorio o web: heterogeneidad de dispositivos, restricciones de batería y de red intermitente, un ciclo de vida gestionado por el sistema operativo (no por el proceso de la propia aplicación), y ventanas de publicación controladas por tiendas de aplicaciones [28]. La decisión entre desarrollo nativo y multiplataforma no es una preferencia estética: tiene consecuencias medibles en rendimiento, mantenibilidad y *time-to-market*, documentadas empíricamente en la literatura de ingeniería de software móvil [29, 30]. La Sección 11 aplica dos de esos criterios cuantitativos —tamaño de artefacto y dependencias de interoperabilidad— directamente sobre el proyecto construido, en vez de citarlos solo como referencia teórica.

8.3. Integración y entrega continuas

La entrega continua se apoya en un *pipeline* automatizado que ejecuta pruebas, empaqueta y despliega en ambientes segregados [31]. La cultura DevOps subyacente exige que ningún cambio llegue a un artefacto publicado sin pasar por los mismos controles [32]: en este proyecto, el pipeline de GitHub Actions (Sección 12) actúa como *quality gate* explícito mediante dependencias declaradas entre *jobs* —si la integración falla, el despliegue de imágenes no ocurre.

8.4. Observabilidad distribuida

La observabilidad de un sistema distribuido se sustenta en tres señales: métricas (agregados numéricos con etiquetas), registros estructurados, y trazas (secuencias de *spans* correlacionados por un identificador común) [33, 34]. El estándar OpenTelemetry [3] unifica su modelo de datos y su exportación, permitiendo que distintos proveedores de *backend* de observabilidad (en este proyecto, Grafana Tempo para trazas y Prometheus para métricas) consuman las mismas señales sin instrumentación duplicada por proveedor.

8.5. Modelo de calidad ISO/IEC 25010:2023

El modelo ISO/IEC 25010:2023 [1] organiza la calidad de producto de software en ocho características —adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad—, cada una descompuesta en subcaracterísticas y evaluable mediante métricas objetivas. Frente a un modelo de calidad puramente cualitativo, esta estructura obliga a declarar de antemano un umbral numérico por característica y a reportar la desviación respecto a ese umbral cuando ocurre, en vez de una valoración subjetiva de si el sistema “es de buena calidad”. La Sección 14 aplica cinco de las ocho características a este proyecto, siguiendo esa misma disciplina de declarar el umbral antes de medir.

8.6. Trazabilidad de temas a artefacto

La Tabla 10 hace explícito, para cada tema de la asignatura cubierto en este documento, el artefacto real del repositorio donde se implementa —no solo el lugar donde se menciona en el texto.

Tabla 10: Trazabilidad de temas de la asignatura a su artefacto en el repositorio

Tema	Artefacto
gRPC	services/telemetry-service/src/main/proto/telemetry.proto y TelemetryGrpcService.java (servidor); cliente real en ZonaVentanaTelemetriaStrategy de svc-principal
Saga por coreografía sobre Kafka	CreateTicketHandler publica TicketCreated; notification-service y report-service (ReportEventListener) reaccionan de forma autónoma, sin orquestador central; TechnicianEventPublisher en auth-service sigue el mismo patrón para TechnicianCreatedEvent
CQRS	Separación por servicio: svc-principal/application/command/ concentra la escritura (CreateTicketHandler, AssignTechnicianHandler, UpdateTicketStatusHandler); report-service/application/ReportQueryService.java concentra la lectura, sobre una proyección propia (TicketSummary)
Fragmentación en CockroachDB	services/svc-principal/.../db/migration/V1__init_ticket_schema.sql (esquema particionado real, migración Flyway versionada); decisión documentada en docs/adr/0003-sharding-policy.md
Spark y ley de Amdahl	spark/ (pipeline paralelo real); análisis en Sección 6 y en la Subsección 3.4 de la Sección 3
Patrón Observer	EscalationLoggingObserver, EscalationMetricsObserver y EscalationEventPublisherObserver en svc-principal; decisión documentada en docs/adr/0005-patrones-gof.md
Arquitectura hexagonal (4 capas)	domain/, application/, infrastructure/ y presentation/ en auth-service, report-service y svc-principal; alcance real declarado en la Sección 9
Observabilidad (Prometheus, Grafana, Tempo, OpenTelemetry)	ops/prometheus/, ops/grafana/, ops/tempo/ y ops/otel-collector/; desarrollado en la Sección 13

9. Arquitectura del sistema

Esta sección documenta la arquitectura resultante tras la Entrega 4 en los tres niveles del modelo C4 [35], y resume las decisiones registradas como *Architectural Decision Records* (ADR) que la sustentan.

9.1. Vista de contexto (C4 Nivel 1)

La Figura 4 sitúa el sistema como una sola caja frente a sus tres tipos de usuario. No hay sistemas externos reales que integrar en este nivel: lo que podría parecer una integración externa —notificaciones multicanal, telemetría de equipos del abonado— está simulado dentro del propio sistema y documentado como tal en cada ADR correspondiente (ver docs/diagrams/c4-nivel1-contexto.md para el detalle de por qué).

Sistema de Soporte Técnico ISP — equipo ACC (Nivel 1: Contexto)



Figura 4: C4 Nivel 1 — contexto del sistema.

9.2. Vista de contenedores (C4 Nivel 2)

El diagrama de contenedores de la Entrega 3 (`docs/diagrams/c4-nivel2-contenedores.md`) se actualiza en esta entrega con las piezas nuevas: las dos aplicaciones cliente (`apps/web`, `apps/mobile`) ya no hablan directamente con cada microservicio por su puerto, sino que pasan por un único `api-gateway` (Spring Cloud Gateway) que enruta por prefijo de *path* sin reescribir el cuerpo de la petición. Se agrega `telemetry-service` (PE-U1, Sección 9.6) como séptimo microservicio, y el stack de observabilidad (OpenTelemetry Collector, Tempo, Prometheus, Grafana, cAdvisor) como un grupo de contenedores transversal. La Figura 5 muestra el diagrama completo.

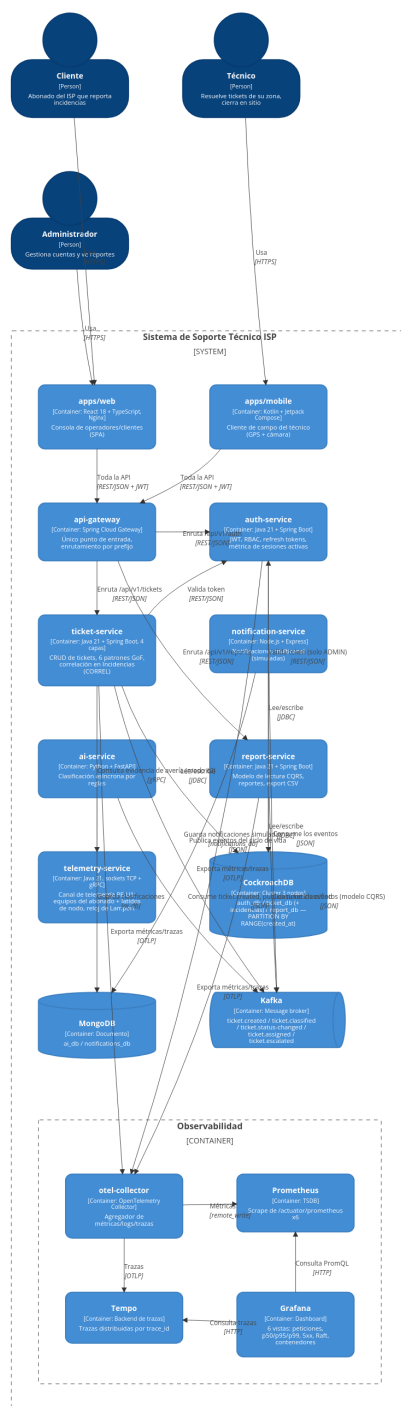


Figura 5: C4 Nivel 2 — contenedores del sistema, Entrega 4.

Tabla 11: Contenedores del sistema tras la Entrega 4.

Contenedor	Tecnología	Responsabilidad
apps/web	React 18 + TypeScript	SPA de consola para clientes/técnicos/administradores
apps/mobile	Kotlin + Jetpack Compose	Cliente de campo para técnicos (APK)
api-gateway	Spring Cloud Gateway	Único punto de entrada, enrutamiento por prefijo
auth-service	Java 21 + Spring Boot	JWT, RBAC, refresh tokens
ticket-service	Java 21 + Spring Boot	CRUD de tickets, arquitectura hexagonal (Sección 9.4)
ai-service	Python + FastAPI	Clasificación asíncrona por reglas
notification-service	Node.js + Express	Notificaciones multicanal simuladas
report-service	Java 21 + Spring Boot	Modelo de lectura CQRS, reportes
telemetry-service	Java 21, sockets TCP + gRPC	Canal PE-U1: equipos del abonado + latidos de nodo, reloj de Lamport (Sección 9.6)
CockroachDB	Cluster 3 nodos	Persistencia distribuida (auth_db/ticket_db/report_db)
Kafka + MongoDB	—	Saga por coreografía, clasificaciones y notificaciones
OTel Collector, Tempo, Prometheus, Grafana, cAdvisor	—	Observabilidad (Sección 13)

9.3. Vista de componentes (C4 Nivel 3)

El único servicio con arquitectura hexagonal completa es `ticket-service` (Sección 9.4), así que es el que se documenta en este nivel del modelo: zoom a sus componentes reales, no a una aproximación. La Figura 6 —fuente versionada en `docs/diagrams/c4-nivel3-componentes.md`— nombra cada caja igual que el árbol de `services/svc-principal/src/main/java/.../ticketservice/`: los seis patrones GoF de la Sección 9.5 son visibles aquí, no solo enumerados en prosa.

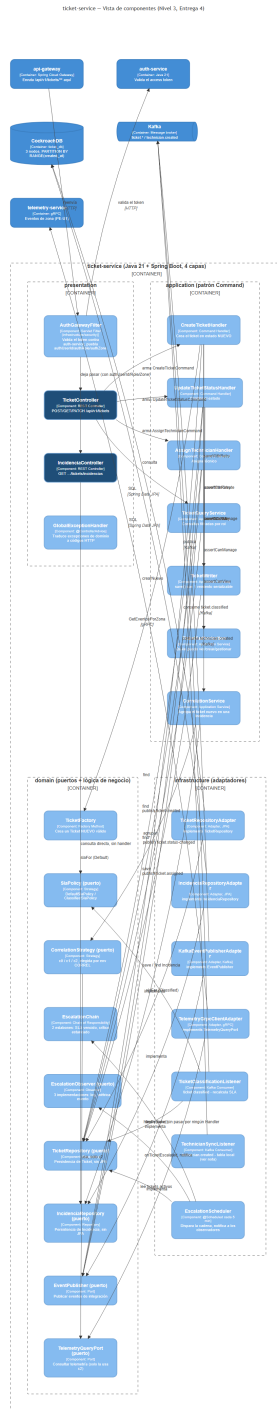


Figura 6: C4 Nivel 3 — componentes de `ticket-service`, Entrega 4.

Dos rutas del diagrama rompen deliberadamente la capa de aplicación, documentado en vez de oculto: `IncidenciaController` consulta `IncidenciaRepository` directo (es una lectura simple, sin caso de uso que orquestar), y `TicketClassificationListener` opera sobre `TicketRepository` sin pasar por ningún `*CommandHandler` (el caso de uso real es aplicar una clasificación asíncrona

llegada por Kafka, no una acción HTTP con autorización de rol). `TechnicianSyncListener` tampoco tiene puerto de dominio: escribe directo en una tabla de sincronización local que no representa un concepto propio de `ticket-service` –es una réplica de solo lectura de datos de `auth-service`, necesaria para que `AssignTechnicianHandler` no viole la clave foránea `tickets_technician_id_fk`.

Construir este diagrama contra el árbol real, y no contra la descripción en prosa que ya existía, encontró además una discrepancia en la Tabla 12: decía que `domain` tiene “cero anotaciones Spring/JPA”. Verificado con `grep` sobre el paquete: era falso para la parte de Spring –12 líneas `import org.springframework.*` en 9 clases (`@Component`, `@Value`, `@Order`, todas para que Spring descubra e inyecte cada implementación de un puerto)– aunque cierto para JPA, que no aparece en ningún archivo de `domain`. Se movieron esas doce líneas a `infrastructure/config/DomainBeansConfig.java` (Sección 9.4), y se agregó una prueba ArchUnit (`architecture/DomainArchitectureTest.java`) que falla la construcción si `domain` vuelve a depender de Spring o de JPA – ya no depende de que alguien lo note leyendo el código a mano.

9.4. Arquitectura hexagonal extendida

El Módulo A de esta entrega exigió refactorizar el microservicio principal en cuatro capas con una regla de dependencia estricta: las capas externas dependen de las internas, nunca al revés. Antes del refactor, una única clase `TicketService` mezclaba acceso a datos vía `JpaRepository` inyectado directamente, la lógica de creación de tickets en línea, dos copias distintas del cálculo de SLA, y los tres casos de uso mutables como métodos sin forma uniforme de interceptarlos.

Tabla 12: Las cuatro capas de `ticket-service` tras el refactor.

Capa	Contenido	Regla
<code>presentation</code>	<code>TicketController</code> , DTOs	Traduce HTTP ↔ comandos; sin lógica de negocio
<code>application</code>	<code>*CommandHandler</code>	Orquesta el caso de uso; no conoce HTTP ni JPA
<code>domain</code>	<code>Ticket</code> , puertos, políticas	Cero anotaciones Spring/JPA, verificado con una prueba ArchUnit que rompe la construcción si deja de serlo (ver Sección 9.3); se prueba con JUnit puro
<code>infrastructure</code>	Adaptadores, filtros, métricas, <i>scheduler</i>	Implementa los puertos del dominio contra CockroachDB, Kafka y HTTP

Una revisión externa del repositorio observó, con razón, que este refactor completo solo se había aplicado a `ticket-service`: los otros cinco microservicios seguían con la organización heredada de la Entrega 2. Se extendió el mismo patrón a `auth-service` y `report-service` –los de menor superficie de los cinco restantes– dejando explícitamente fuera de alcance `ai-service`, `notification-service` y `api-gateway` por el tiempo disponible antes del cierre; se documenta el alcance real en vez de sugerir un refactor completo que no ocurrió.

Tabla 13: Alcance real del refactor a cuatro capas, y la violación concreta que corrigió en cada caso.

Servicio	Estado	Violación real corregida
ticket-service	4 capas + 6 patrones GoF	TicketService mezclaba JPA, SLA y los tres casos de uso en una sola clase (ADR-0005)
auth-service	4 capas	User/RefreshToken eran entidades JPA directamente en el paquete domain; AuthService dependía de JpaRepository sin un puerto intermedio
report-service	4 capas	ReportController llamaba a TicketSummaryRepository (JPA) y filtraba en memoria directamente en la capa de presentación, sin capa de aplicación
ai-service, notification-service, api-gateway	Sin cambios	Fuera de alcance en esta entrega (declarado explícitamente, no un olvido)

Ambos refactores se verificaron de punta a punta contra el sistema real, no solo con pruebas unitarias: se levantó cada servicio con su base CockroachDB y Kafka reales, y se ejercitaron en vivo los flujos completos (**auth-service**: registro, *login*, refresco con rotación de token, y los tres rechazos reales –401 sin token, 401 credenciales inválidas, 403 sin rol–; **report-service**: `/summary`, `/tickets` filtrado y `/export.csv` contra datos reales ya existentes, más los mismos dos rechazos). Las 15 pruebas unitarias de **auth-service** y las 11 de **report-service** pasan en verde tras el refactor, sin haber cambiado ningún comportamiento observable del sistema.

9.5. Seis patrones GoF

Se implementaron seis patrones del catálogo de Gamma et al. [2] —la unión del mínimo exigido por el Módulo A (Repository, Factory Method, Strategy) y del conjunto asignado específicamente al equipo ACC en la Tabla 1 de la guía de la entrega (Command, Chain of Responsibility, Observer, Repository, Strategy)—, cada uno documentado en detalle como [ADR-0005](#) con el problema de diseño real que resolvía antes del refactor:

1. **Repository** — desacopla el dominio de `JpaRepository` (Spring Data) vía un puerto (`TicketRepository`, en el paquete `domain`) implementado por un adaptador de infraestructura.
2. **Factory Method** — centraliza la creación de un `Ticket` nuevo (`TicketFactory.crearNuevo`), antes duplicada en línea.
3. **Strategy** — dos políticas de SLA intercambiables (`DefaultSlaPolicy`, `ClassifiedSlaPolicy`) que antes vivían hardcodeadas en dos lugares distintos.
4. **Command** — cada acción mutable (crear, cambiar estado, asignar técnico) es un objeto inmutable ejecutado por un *handler* dedicado, desacoplando al controlador de cómo se ejecuta la acción.
5. **Chain of Responsibility** — dos eslabones de evaluación de escalado (`SlaBreachEscalationHandler`, `StaleCriticalEscalationHandler`) que implementan una funcionalidad que antes no existía en absoluto: el estado `ESCALADO` era inalcanzable desde la Entrega 3.
6. **Observer** — tres observadores del evento de escalado (*logging*, métrica de negocio, publicación en Kafka), agregables sin modificar el sujeto que los notifica.

Cada patrón reemplaza una duplicación o acoplamiento verificable contra el historial de `TicketService.java` previo a esta entrega, no una envoltura artificial agregada solo para completar el conteo mínimo.

9.6. Canal de telemetría PE-U1 y correlación de tickets en Incidencias

Se construyó `telemetry-service`: un servidor de sockets TCP que recibe reportes periódicos de equipos del abonado y latidos de los 3 nodos del clúster, les asigna un *timestamp* de orden causal mediante un reloj lógico de Lamport [36], y expone el registro resultante por gRPC (`GetEventosPorZona`), ordenado por ese *timestamp* y no por orden de llegada (ADR-0007). Dos clientes simuladores, cada uno con su propio reloj de Lamport independiente del servidor, demuestran la regla clásica $\max(\text{local}, \text{recibido}) + 1$ entre procesos reales.

Sobre ese canal se construyó el mecanismo de correlación de tickets en *Incidencias*: un ticket nuevo puede agruparse con otros de la misma zona si la estrategia activa (variable de entorno `CORREL`, patrón Strategy) lo decide. Se implementaron tres estrategias intercambiables (ADR-0008), seleccionadas por Spring inyectando un `Map<String, CorrelationStrategy>` y resolviendo la activa por el nombre exacto del *bean*:

- `c0` (línea base): cada ticket abre su propia incidencia.
- `c1`: se une a la incidencia abierta más reciente de la misma zona dentro de una ventana deslizante, ciega a la telemetría.
- `c2`: mismo criterio de `c1`, más una consulta real por gRPC a `telemetry-service`; solo agrupa si hay evidencia real de telemetría en esa ventana. Si el canal no responde, el ticket queda aislado en su propia incidencia – nunca bloquea ni revierte la creación del ticket (mismo principio de fallo abierto que ya aplica el *publish* de Kafka, ADR-0004).

Ninguna de las tres estrategias está diseñada para resolver el caso de dos averías reales golpeando la misma zona en la misma ventana – ambas pueden fundirlas en una sola incidencia. Es intencional: el objetivo de un escenario con dos averías simultáneas en el protocolo experimental es *revelar* ese error, no demostrar que no ocurre. Se construyó además un inyector de averías (`experimentos/injector_averias.py`) que provoca una avería simulada real – crea tickets reales vía la API y envía telemetría real al canal – y deja constancia exacta de a quién afectó en `verdad_campo.csv`, precisamente porque en un ISP real nadie sabe con certeza qué abonados estaban afectados por una avería y aquí sí se conoce con exactitud. Una primera corrida manual del inyector junto con el script de análisis (`experimentos/analizar_correlacion.py`) confirmó el mecanismo funcionando de punta a punta con datos reales: 100 % de precisión y exhaustividad en el agrupamiento bajo `c2` con telemetría real corroborando la avería. La batería estadística completa del protocolo (10 repeticiones $\times$ 4 escenarios $\times$ 3 modos) es trabajo de cierre pendiente, no incluido en esta corrida de verificación.

9.7. Postura de consistencia distribuida

La Entrega 3 estableció (ADR-0004) una postura CAP/PACELC [10, 11] mixta por agregado de dominio: la creación de tickets prioriza disponibilidad (AP), mientras que los datos de SLA y facturación exigen consistencia fuerte (CP). CockroachDB no ofrece un modo “eventual” configurable por tabla —su consistencia serializable por defecto, lograda mediante consenso Raft [8], cubre el caso CP sin configuración adicional; la postura AP se aproxima evitando reintentos automáticos silenciosos en las operaciones críticas. Esta postura sigue vigente sin cambios en la Entrega 4: el refactor a arquitectura hexagonal reorganizó *dónde* vive la lógica de persistencia, no *cómo* se comporta frente a particiones de red.

10. Aplicación web

10.1. Framework y justificación

Se eligió React 18 con TypeScript en modo estricto sobre Vite, empaquetado en un *Dockerfile multi-stage* (etapa **builder** con Node LTS, etapa de ejecución con **nginx:alpine**). La aplicación sigue el patrón de **arquitectura por características** (*feature-based*): cada módulo de negocio (**console**, **admin**, **reports**, **auth**) agrupa sus propios componentes, *hooks* de acceso a la API y pruebas, en vez de una organización por tipo técnico (todos los componentes juntos, todos los servicios juntos) que dificulta ubicar el código relacionado con una misma funcionalidad de negocio.

10.2. Capturas reales

La Figura 7 muestra cuatro pantallas capturadas en vivo contra el *stack* completo en ejecución (no maquetas): inicio de sesión, la consola de operadores con datos reales (incluido un ticket creado en vivo vía la API durante esta misma verificación, ver **docs/evidencias/**), el panel de reportes (modelo de lectura CQRS de **report-service**, con un total distinto al de la consola porque se reconstruye de forma asíncrona a partir de eventos de Kafka, no es una copia sincrónica), y la administración de cuentas.

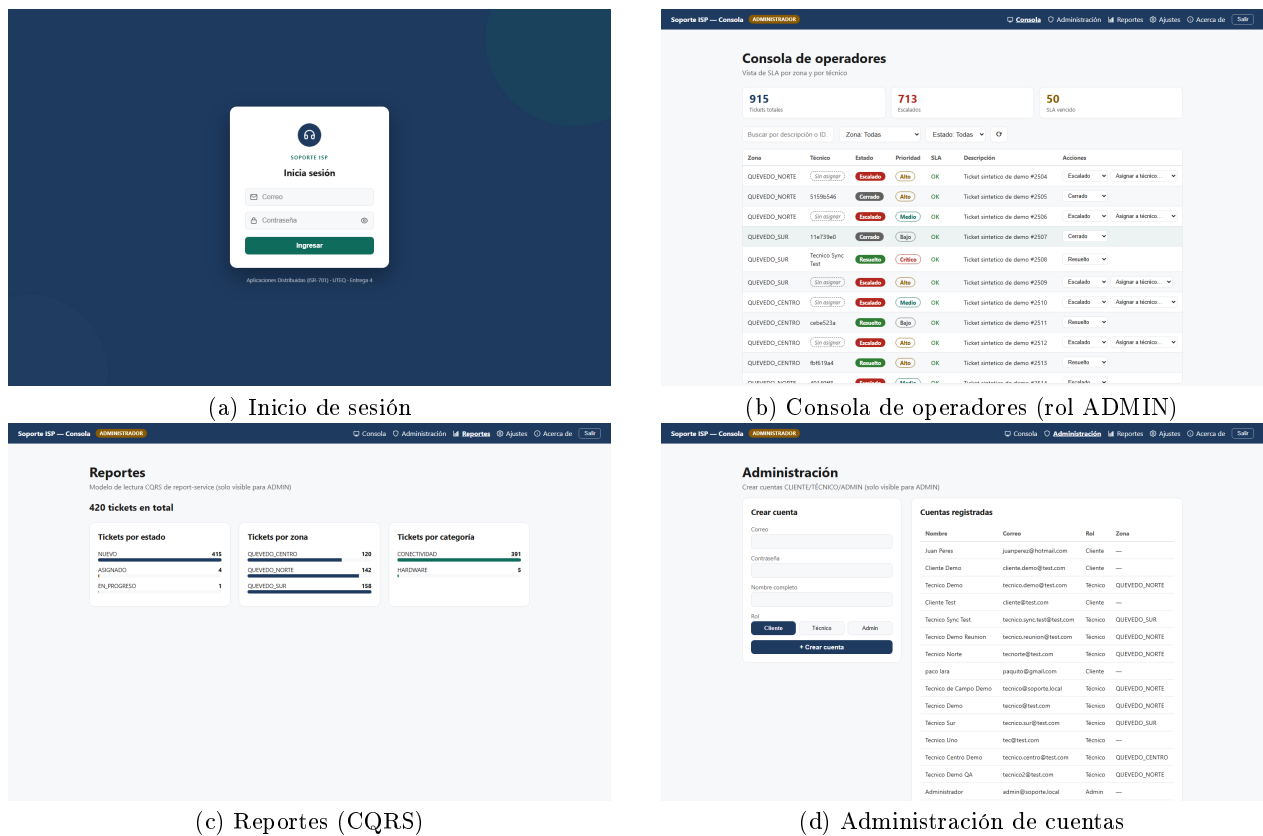


Figura 7: Capturas reales de **apps/web** contra el sistema en ejecución (2026-09-03).

10.3. Rutas y control de acceso

La aplicación cubre las cinco rutas mínimas exigidas por el Módulo B (`/`, `/login`, `/main`, `/settings`, `/about`), más dos adicionales con acceso restringido por rol (`/admin`, `/reports`). El enrutamiento protegido se implementa con dos componentes de orden superior: `ProtectedRoute` (exige sesión válida) y `RoleRoute` (exige, además, un rol específico), ambos cubiertos por pruebas unitarias (`ProtectedRoute.test.tsx`, `RoleRoute.test.tsx`).

El JSON Web Token [4] se guarda en `sessionStorage`, nunca en `localStorage` sin cifrar: se pierde al cerrar la pestaña del navegador, acotando la ventana de exposición ante un ataque de *cross-site scripting* que lograra ejecutar código en el origen de la aplicación.

10.4. Internacionalización y estados explícitos

La interfaz está completamente traducida a español e inglés con `react-i18next`, y maneja de forma explícita los tres estados que toda pantalla que consume datos remotos debe distinguir: carga (esqueletos animados en la tabla), error (mensaje con icono, nunca una pantalla en blanco) y vacío (mensaje distinto a error cuando la consulta es válida pero no hay resultados).

La consola es **consciente del rol**: un usuario con rol `CLIENTE` ve un título y subtítulo orientados a su propio caso de uso (“mis solicitudes”, no jerga interna de operaciones), y en la columna de técnico asignado ve un distintivo genérico (`AssignedChip/UnassignedChip`) en vez del identificador interno del técnico, que no le aporta información accionable y expondría un identificador de otro usuario innecesariamente.

10.5. Detalle completo del ticket

Al hacer clic en cualquier fila de la tabla se abre un modal (`TicketDetailModal`) que trae el recurso completo desde el backend (no solo los campos ya presentes en la fila), incluyendo la línea de tiempo (creación, plazo de SLA, resolución si aplica) y el identificador completo del ticket en un elemento seleccionable —pensado explícitamente para poder buscar ese mismo identificador después, por ejemplo tras cerrarlo desde la aplicación móvil (Sección 11), y confirmar visualmente que ambos clientes leen el mismo estado del mismo backend.

10.6. Calidad

`eslint` con las reglas por defecto de `@typescript-eslint` corre en modo estricto (`-max-warnings 0`: cualquier advertencia rompe la compilación, no solo los errores). La cobertura de pruebas (Vitest + Testing Library) alcanza el 87.6 % de líneas sobre el total de la aplicación, con la única brecha significativa en el componente de detalle recién agregado (`TicketDetailModal.tsx`, 34.8 %), pendiente de cubrir antes del cierre del proyecto.

11. Aplicación móvil

11.1. Justificación cuantitativa de Android nativo

El Módulo C permite dos rutas para el cliente móvil obligatorio: Android nativo (Kotlin + Jetpack Compose) o multiplataforma (Flutter 3.x o React Native 0.74+). El Anexo B de la guía de esta entrega exige justificar esa elección con al menos dos criterios cuantitativos, no solo preferencia estética —conforme a lo discutido en la literatura sobre desarrollo de aplicaciones móviles [28, 29, 30]. El cliente móvil de este proyecto es para técnicos de campo: usa cámara y geolocalización de forma

continúa durante la jornada, en dispositivos de gama variable asignados por la operadora. Se eligió **Android nativo con Kotlin + Jetpack Compose** (MVVM, inyección de dependencias manual), sustentado en dos mediciones directas sobre el proyecto ya construido (ADR-0006):

1. **Tamaño del artefacto instalable:** el APK de depuración pesa **11.06 MB**, sin ningún motor de renderizado embebido adicional —Jetpack Compose compila a las mismas vistas nativas de Android (*bytecode* ART), a diferencia de un framework multiplataforma que empaqueta su propio motor (Skia/Flutter engine, o el puente JavaScript de React Native) dentro de cada instalación.
2. **Dependencias de terceros para las capacidades obligatorias:** cámara y geolocalización se resuelven con **cero dependencias de interoperabilidad** —la cámara usa `ActivityResultContracts.TakePicture` (parte del propio SDK de Android) y la geolocalización una única dependencia oficial de Google (`play-services-location`). Un framework multiplataforma necesitaría, en cambio, un *plugin* puente con la sobrecarga de serialización y el riesgo de mantenimiento que eso implica.

Como criterio cualitativo adicional: el backend del proyecto ya es JVM (Java 21 + Spring Boot), por lo que Kotlin comparte máquina virtual y herramientas de *build* (Gradle) con el resto del equipo, reduciendo el salto conceptual frente a adoptar Dart o un puente nativo de JavaScript desde cero.

11.2. Arquitectura MVVM y modo sin conexión

Cada pantalla sigue el patrón **MVVM**: un `ViewModel` concentra el estado (`StateFlow`) y la lógica, mientras la vista en Jetpack Compose solo observa y dibuja. El listado de tickets asignados usa una caché local con Room, de forma que un técnico en campo sin cobertura de red siga viendo su carga de trabajo del día.

11.3. Capacidades del dispositivo

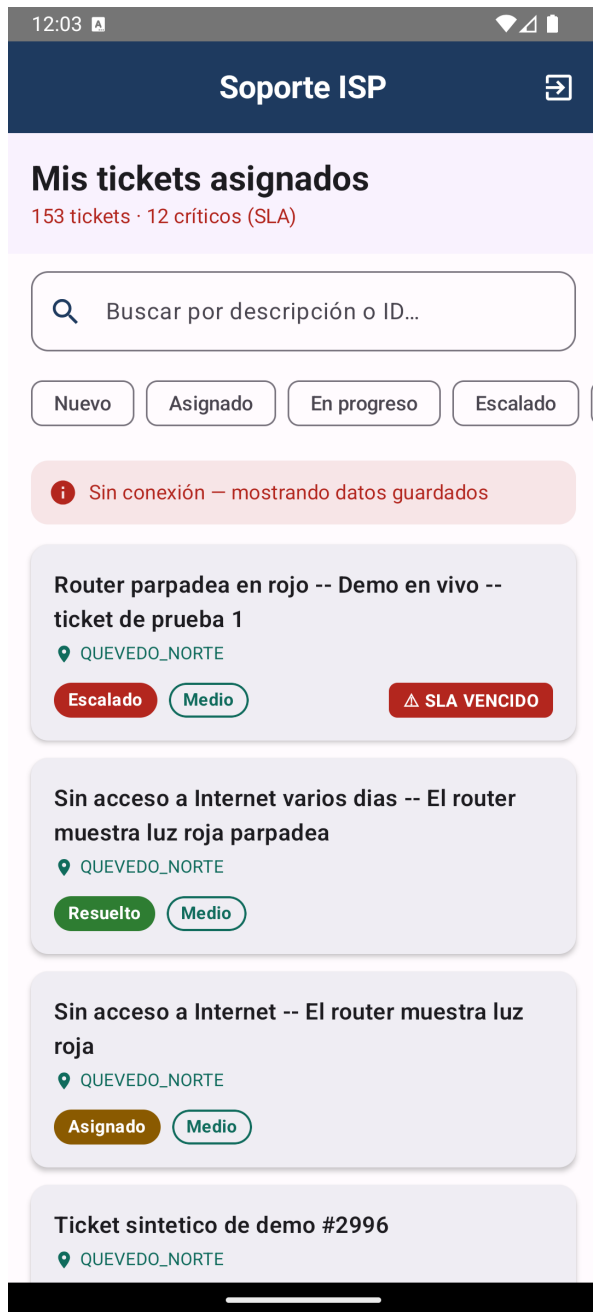
Se integran las dos capacidades obligatorias del Módulo C:

- **Cámara:** evidencia fotográfica del cierre de un ticket, capturada con `ActivityResultContracts.TakePicture` y un `FileProvider` propio de la aplicación (sin escribir a almacenamiento externo compartido).
- **Geolocalización:** ubicación del cierre capturada con `FusedLocationProviderClient`, solicitada solo en el momento del cierre, no como rastreo continuo en segundo plano, y precedida de un diálogo explícito de consentimiento que declara la finalidad y el destino del dato antes de pedir el permiso del sistema operativo (`TicketDetailScreen.kt`).

La foto y las coordenadas no se quedan en el teléfono: `TicketRepository.closeOnSite` las envía junto con el cambio de estado a **RESUELTO** en una sola llamada `PATCH /api/v1/tickets/{id}/status` (foto codificada en Base64), con reintento —hasta 3 veces ante `IOException`, con espera de 1 y 2 s— para tolerar una red de campo inestable. El servicio principal las guarda como columnas `BYTES/FLOAT8` adicionales de la fila del ticket (migración `V3__add_close_evidence.sql`), sin un almacén de objetos aparte. El reintento es idempotente: si el primer `PATCH` sí llegó al servidor y solo se perdió la respuesta, el segundo no vuelve a mover `resolvedAt` ni a recalcular el incumplimiento de SLA, y el servidor exige la evidencia cuando quien cierra es un **TECNICO** (un **ADMIN** puede seguir resolviendo un ticket sin pasar por el móvil).

La Figura 8 muestra ambas capacidades reales en el emulador (`Medium_Phone_API_36.0`), instalando el mismo APK de depuración que produce el *job* `build-mobile-apk` de CI/CD: el listado (que además exhibe en vivo el modo sin conexión de la Sección anterior —el emulador arrancó con datos ya cacheados en Room, y la aplicación lo declara explícitamente en vez de mostrar una lista

vacía o congelarse) y la pantalla de detalle con los botones reales **Tomar Foto** y **GPS Cierre**.



(a) Listado (modo sin conexión, datos cacheados con Room)



(b) Detalle: cámara y GPS de cierre

Figura 8: Capturas reales de `apps/mobile` en emulador Android (2026-09-03).

La pantalla de detalle muestra además el identificador completo del ticket en un elemento de texto seleccionable —una decisión deliberada para poder demostrar, en una presentación en vivo, que la aplicación móvil y la aplicación web leen y escriben sobre el mismo backend real: cerrar un ticket desde el teléfono y encontrarlo actualizado en la consola web buscando por ese mismo identificador (Sección 10).

11.4. Pruebas

La capa de `ViewModel` tiene pruebas unitarias con JUnit 5 y `kotlinx-coroutines-test` (`LoginViewModelTest`). La prueba instrumentada de extremo a extremo exigida por el Módulo C (Espresso o Compose Testing) cubre el flujo de validación de formulario de inicio de sesión (`LoginScreenTest`, Compose Testing) contra la actividad real de la aplicación, sin depender de que el backend esté disponible —de forma estable en un pipeline de integración continua.

12. Pruebas y CI/CD

12.1. Pirámide de pruebas

La estrategia de pruebas cubre las cuatro capas clásicas de la pirámide [37], más una capa de carga, cada una respondiendo una pregunta que las demás no pueden:

Tabla 14: Pirámide de pruebas del sistema, con resultados reales.

Capa	Herramienta	Pregunta que responde	Resultado
Unitaria	JUnit 5 / Vitest / JUnit Android	¿Esta clase, aislada, hace lo que dice?	315/315 backend (5 servicios Java + Node + Python), 128/128 web, 33/33 móvil
Integración	Testcontainers	¿Funciona contra un CockroachDB real, no un <i>mock</i> ?	<code>TicketRepositoryIntegrationTest</code>
Contrato	Pact (consumidor + proveedor)	¿El backend sigue la forma exacta que el cliente espera?	2/2 consumidor, 2/2 proveedor
E2E	Playwright / Compose Testing	¿Todo el camino, navegador real incluido, funciona?	9/9 en Chromium+Firefox+WebKit
Carga	Locust	¿Aguanta concurrencia real?	Sección 14

Cada cifra de la fila «Unitaria» se obtiene ejecutando la suite correspondiente, no contándola a mano: `mvn test` dentro de `services/auth-service`, `services/svc-principal`, `services/report-service`, `services/api-gateway` y `services/telemetry-service` para los cinco servicios Java (63, 161, 37, 8 y 15 pruebas respectivamente); `npm test` (que invoca `node -test`) dentro de `services/notification-service` (15); `pytest` dentro de `services/ai-service` (16) — total backend, 315; `npm test` (que invoca `vitest run`) dentro de `apps/web` (128); y `./gradlew :app:testDebugUnitTest` dentro de `apps/mobile` para la capa unitaria móvil (33). Son los mismos comandos, sin variación, que ejecutan los trabajos `test-backend`, `test-web` y `test-mobile` de la Tabla 15, así que la cifra publicada es la que produce el propio flujo automático en cada envío, no una recontada aparte.

La fila «Unitaria» no es una población disjunta de las filas «Integración» y «Contrato»: los 315 de backend incluyen las 2 pruebas de `TicketRepositoryIntegrationTest` (Testcontainers, la misma clase que reporta la fila «Integración», porque `mvn test` las ejecuta en el mismo módulo) y *no* incluyen la prueba de proveedor Pact de `svc-principal`, que `mvn test` omite con `@EnabledIfSystemProperty` salvo que se invoque explícitamente (`-Dpact.verifier.publishResults=true`), y que la fila «Contrato» sí cuenta por separado al correrla así. Los 128 de web incluyen las 2 pruebas de consumidor Pact (`apps/web/tests/contract/ticketsApi.pact.test.ts`), que la fila «Contrato»

vuelve a publicar como «2/2 consumidor». Se documenta así, en vez de depurar cada fila para que sea disjunta de las demás, porque las cifras de «Unitaria» son literalmente lo que produce `mvn test/npm test` sin banderas adicionales —el mismo comando que corre la CI—, y alterar esa cifra para excluir pruebas que técnicamente conviven en el mismo módulo dejaría de corresponder a la salida real del comando publicado.

`api-gateway` y `telemetry-service` tenían `Dockerfile` y pruebas JUnit reales en el repositorio, pero ningún trabajo del flujo los ejecutaba ni los publicaba (hallazgo documentado en el commit `2c5bba5`, fuera de alcance de esa corrección en su momento). De las 8 pruebas nuevas de `api-gateway`, 7 son de `HttpMetricsGlobalFilter`: el único filtro del *gateway* sin ejercitar, cuya lógica reduce identificadores UUID en la ruta a un marcador de posición (`/api/v1/tickets/{id}`) para no romper la cardinalidad de las métricas de Prometheus.

`AuthGatewayFilter` tampoco tenía ninguna prueba, a pesar de ser el único punto que exige autenticación sobre `/api/v1/tickets/**`: valida el token delegando en `auth-service` por HTTP en vez de verificar la firma localmente (Sección 12.1.1). Sus 12 pruebas nuevas corren contra un doble real de `auth-service` (un `com.sun.net.httpserver.HttpServer` local, no un *mock* de la biblioteca HTTP), incluida la caída completa del servicio: con el doble apagado a mitad de la prueba, el filtro falla cerrado (401), nunca deja pasar. `HttpMetricsFilter` —la misma clase, copiada sin variación en `auth-service`, `svc-principal` y `report-service`, ver el comentario de la propia clase— tampoco tenía pruebas en ninguna de sus tres copias; las 12 nuevas (4 por servicio) documentan además un hallazgo real: a diferencia de `HttpMetricsGlobalFilter` en el *gateway*, que normaliza cualquier UUID en la ruta sin condición, esta clase solo normaliza cuando Spring MVC ya resolvió un *handler* —una petición que no *matchea* ninguna ruta cae a la URI cruda sin normalizar, la misma cardinalidad sin límite que la clase existe para evitar. Se deja caracterizado, no corregido, para que quien lo arregle tenga la prueba lista para actualizar junto con el fix.

`JwtAuthenticationFilter`, en `auth-service`, tampoco tenía prueba propia: la clase que puebla el `SecurityContext` a partir de un token —la pieza que hace que `@PreAuthorize("hasRole('ADMIN')")` funcione en `AdminController`— es distinta de `JwtService` (la emisión y validación del JWT en sí, que ya tenía prueba propia desde antes). Sus 7 pruebas nuevas usan un `JwtService` real, no simulado, e incluyen el caso que este filtro más necesita rechazar: un token con la forma correcta pero firmado con un secreto distinto al real. `report-service` tiene su propia copia de `AuthGatewayFilter` —no idéntica a la de `ticket-service`: agrega una restricción propia (solo el rol ADMIN puede consultar reportes, aunque el token sea válido) que no existe en ningún otro servicio, así que necesitaba su propia prueba, no la del hermano de `svc-principal`. Sus 8 pruebas nuevas confirman que un TECNICO o CLIENTE con sesión real reciben 403 (prohibido), no 401 (no autenticado) —son errores distintos que un cliente del API necesita distinguir.

`AdminBootstrap` (`auth-service`) crea la cuenta ADMIN por defecto en el primer arranque, únicamente si todavía no existe ninguna —el guardia exacto que evita crear un ADMIN duplicado en cada reinicio— y no tenía prueba. Sus 2 pruebas nuevas cubren ambas ramas: sin ADMIN previo, se crea uno con la contraseña cifrada (nunca el texto plano de configuración); con uno ya existente, no se guarda nada. `ReportQueryService.filtered()` —la única lógica propia de ese servicio, el resto son delegaciones directas al puerto— filtra en memoria por hasta tres criterios independientes, opcionales e insensibles a mayúsculas, y tampoco tenía prueba; sus 8 pruebas nuevas cubren cada filtro por separado, la combinación de los tres a la vez, y el caso que un filtro aplicado por error con OR en vez de AND dejaría pasar sin que nadie lo notara.

`TicketListViewModel` (`apps/mobile`) tampoco tenía prueba: es el único *ViewModel* que combina dos filtros independientes —búsqueda por texto y estado— sobre el mismo flujo, con una distinción sutil que solo una prueba puede proteger de verdad: el total y el conteo de incumplimientos de SLA se calculan sobre *todos* los tickets, no sobre los ya filtrados, así que el resumen sigue siendo

correcto aunque la lista visible esté filtrada. Sus 6 pruebas nuevas usan Turbine para suscribirse al `StateFlow` —a diferencia de `TicketDetailViewModel` (un `MutableStateFlow` simple), este expone su estado vía `combine(...).stateIn(..., WhileSubscribed(5_000), ...)`, que solo empieza a producir valores cuando alguien lo está observando— y cubren, además de la distinción de conteos, que la búsqueda no distingue mayúsculas, que los dos filtros se combinan con `AND`, que tocar el mismo filtro de estado dos veces lo quita, y que un fallo de red al refrescar conserva los tickets ya guardados en caché en vez de vaciar la pantalla.

Cuatro piezas más quedaron sin prueba fuera del backend Java. `kafka_consumer.py` (`ai-service`): `classify()` ya tenía prueba propia, pero no `_handle_ticket_created()` (el guardado en Mongo y la publicación de `ticket.classified`) ni `_consume_loop()` —sus 4 pruebas nuevas confirman, entre otras cosas, que un mensaje de Kafka malformado no tumba el hilo consumidor para siempre, tal como promete el comentario del propio archivo. `kafkaConsumer.js` (`notification-service`): `dispatch()` ya tenía prueba, pero no `handleMessage()` (el mapeo a documentos de Mongo); sus 3 pruebas nuevas confirman que un tópico sin notificaciones que despachar ni siquiera abre la colección. `apps/web`: el interceptor que adjunta `Authorization: Bearer` a cada petición (`apiClient.ts`) nunca se había probado en aislamiento —`authApi.test.ts` simula `axios.post` entero, sin pasar por la cadena real de interceptores—; sus 3 pruebas nuevas invocan el interceptor registrado directamente. Y `useAuth` (el *hook* de autenticación) se usaba de paso en `ProtectedRoute.test.tsx` pero nunca se ejercitaban `login()/logout()` ni los estados `loading/error`; sus 7 pruebas nuevas cubren el flujo completo, incluido un rechazo que no es una instancia de `Error` (la rama que cae al mensaje genérico).

Dos piezas más, una en cada cliente, seguían sin prueba propia. `TicketDetailModal` (`apps/web`): pide de nuevo el ticket al backend en vez de reusar el objeto de la fila —para no mostrar un estado desactualizado si `EscalationScheduler` lo cambió entre que se cargó la lista y el clic— y tiene una guardia de cierre (clic en el fondo cierra el modal, clic dentro del contenido no) fácil de romper sin que ninguna prueba lo note; sus 9 pruebas nuevas cubren el estado de carga, el de error, y confirman explícitamente que un clic dentro del contenido *no* dispara `onClose`. `TicketRepository` (`apps/mobile`): la pieza con más lógica de negocio propia de toda la app móvil —el reintento de `closeOnSite` (solo ante fallo de *red*, nunca ante un 4xx/5xx real) y el patrón caché-primero de `get-Ticket`— solo se ejercitaba indirectamente a través de `TicketDetailViewModelTest`, que mockea el repositorio entero y nunca corre su lógica real. Sus 9 pruebas nuevas mockean `android.util.Base64` de forma estática (no disponible en pruebas JVM puras, razón por la que ningún otro archivo lo había tocado hasta ahora) y confirman, entre otras cosas, que un error HTTP real no dispara reintento —a diferencia de un fallo de red, que sí— y que agotar los 3 intentos sin conexión propaga la excepción real, no una genérica.

Dos piezas más, pequeñas pero reales. El manejador de `GET /api/v1/notifications` (`notification-service`) decide su filtro de Mongo según si llega `ticketId` por query string o no —«si viene, filtra; si no, trae todo»— y no tenía prueba; sus 4 pruebas nuevas invocan el manejador real extraído del router de Express (no hay `supertest` instalado en este proyecto), sin levantar un servidor HTTP, y confirman también que el campo `_id` crudo de Mongo nunca sale en la respuesta. `SessionMetrics` (`auth-service`) registra el *Gauge* `app_active_sessions` exigido por el Módulo F de la Guía de Entrega 4, pero nada confirmaba que quedara registrado con ese nombre exacto ni que reflejara el conteo real del repositorio; sus 3 pruebas nuevas usan un `SimpleMeterRegistry` real y comprueban, entre otras cosas, que el *Gauge* se recalcula en cada lectura y no queda cacheado con el primer valor.

Dos piezas más en `svc-principal`. `RepositoryTimingAspect` —el único punto de instrumentación del histograma `crdb_query_duration_seconds`, vía AOP sobre todo el paquete de persistencia— no tenía prueba, y su garantía central (el cronómetro vive en un bloque `finally`) nunca se había verificado: sus 4 pruebas nuevas confirman que una consulta que falla igual cuenta para

el histograma, y que la excepción real sigue propagándose sin que el aspecto la trague. **TicketMapper** —la traducción campo a campo entre el dominio puro y su entidad JPA— tampoco tenía prueba, pese a que un mapeo así es exactamente el tipo de código que se rompe en silencio cuando se agrega un campo a un lado y se olvida el otro: ya pasó de verdad con los campos de evidencia (`evidencePhoto/evidenceLatitude/evidenceLongitude`, Entregable 10), agregados a **Ticket** y **TicketJpaEntity** por separado. Sus 4 pruebas nuevas incluyen un viaje de ida y vuelta campo por campo con `usingRecursiveComparison()`, que fallaría de inmediato si un campo nuevo se agregara a un lado sin el otro.

TicketController no tenía prueba propia —sus manejadores sí, extensamente—, pero tiene una pieza de lógica que le pertenece solo a él: decodificar en Base64 la evidencia fotográfica del cierre en sitio. Sus 7 pruebas nuevas (manejadores mockeados, sin *MockMvc* ni contexto de Spring) documentan un hallazgo real, no solo lo prueban: un `evidencePhotoBase64` malformado —un error del lado del cliente móvil, no necesariamente malicioso— no tiene manejo explícito y cae al `@ExceptionHandler(Exception.class)` genérico de `GlobalExceptionHandler`, que responde 500 «Error interno» en vez de un 400 claro. Se deja caracterizado, no corregido.

El único *endpoint* HTTP de **ai-service** —`GET /api/v1/ai/classifications/{ticketId}`, la contraparte de lectura de lo que `_handle_ticket_created()` escribe en Mongo— tampoco tenía prueba propia. Sus 4 pruebas nuevas (llamando la función directo, sin **TestClient/httpx**, que ni siquiera está en `requirements.txt`) confirman que un ticket sin clasificación todavía responde 404 con el identificador en el mensaje —en vez de un 500 o, peor, un 200 con cuerpo vacío— y que la proyección `{"_id": 0}` realmente se pasa a Mongo, no solo se intenta: un `ObjectId` crudo filtrado al cliente rompería la serialización JSON de FastAPI en cuanto existiera al menos una clasificación guardada.

AuthController, en cambio, no tenía ninguna prueba propia pese a exponer los cinco *endpoints* de sesión del sistema. Sus 7 pruebas nuevas (con **AuthService** mockeado, sin *MockMvc* ni contexto de Spring) verifican que `login` extrae `email/password` del *request* en vez de reenviarlo entero, y sobre todo cubren la única lógica que le pertenece al controlador y no al servicio: `validate()` parsea el encabezado **Authorization** a mano (en vez de dejarlo a Spring Security, ver comentario en la clase) para que un *token* ausente o mal formado devuelva un 401 con el envoltorio **ApiResponse** del dominio en lugar del manejador genérico del framework; un encabezado nulo y uno sin el prefijo `"Bearer "` quedan probados por separado.

Las tres estrategias de correlación de incidencias —`c0/c1/c2`, el componente exacto que el Escenario 4 del protocolo experimental (dos averías simultáneas en la misma zona) pone a prueba (ver ADR-0008)— ya tenían una suite propia (**CorrelationStrategyTest**) desde que se introdujeron, pero con dos huecos concretos. Ninguna prueba de `c1` confirmaba que los tickets que ya estaban en una incidencia sobrevivieran al unirse uno nuevo —solo se verificaba que el recién llegado entrara, no que los anteriores no se perdieran, un bug de reemplazo-en-vez-de-agregar habría pasado inadvertido—; y ninguna prueba de `c2` verificaba el valor real que la conversión minutos-a-segundos de la ventana le pasa al puerto de telemetría, porque los dobles de **TelemetryQueryPort** usados ignoraban ese parámetro por completo. Sus 2 pruebas nuevas cierran ambos huecos: una confirma que unirse a una candidata agrega el ticket sin descartar los que ya estaban, y la otra captura el argumento real recibido por el puerto y confirma que una ventana de 15 minutos configurada llega como 900 segundos, ni un valor crudo sin convertir ni otra escala.

Dos piezas más, una en cada extremo del sistema. **TicketSummaryMapper** (**report-service**) —el mismo tipo de traducción campo a campo entre dominio puro y entidad JPA que ya causó un bug real en **ticket-service** (ver **TicketMapper**, Entregable 10)— tampoco tenía prueba, con un agravante propio: **TicketSummary** solo se reconstruye a partir de eventos de Kafka (ver **ReportEventListener**), así que un campo perdido en el mapeo no se nota en una escritura directa a la base, se nota semanas

después, como un reporte con un dato en blanco que nadie relaciona con el mapeo. Sus 4 pruebas nuevas incluyen el mismo viaje de ida y vuelta con `usingRecursiveComparison()` que ya protege a `TicketMapper`. `AuthRepository` (`apps/mobile`) tampoco tenía prueba: es el único punto que decide si un login exitoso en el servidor realmente cuenta como sesión iniciada en el cliente, envolviendo la llamada en `runCatching` y exigiendo `data` no nulo con `requireNotNull` antes de guardar los tokens. Sus 5 pruebas nuevas documentan, entre otras cosas, que una respuesta HTTP 200 con `data` nulo —un caso real y no solo teórico, dado que `ApiResponse.data` es nullable en el contrato— falla en vez de guardar tokens vacíos y reportar una sesión iniciada que en realidad no existe.

Dos piezas más, ambas en el extremo de PE-U1 que alimenta la correlación de incidencias. `TelemetryStore` (`telemetry-service`) —el *buffer* en memoria del que lee `ZonaVentanaTelemetryStrategy` (c2, ver Ronda 12 más arriba) para decidir si hay evidencia real de avería antes de agrupar tickets— tampoco tenía prueba propia: las pruebas de c2 mockean `TelemetryQueryPort` por completo y nunca ejercitan este buffer real. Sus 7 pruebas nuevas confirman, entre otras cosas, la garantía que el propio comentario de la clase remarca —los eventos se devuelven ordenados por *timestamp* de Lamport (orden causal), no por orden de llegada— registrando eventos deliberadamente fuera de ese orden para que una implementación que solo devolviera la lista tal cual, o la ordenara por hora de recepción, no pasara la prueba; también confirman que `purgar()` descarta eventos vencidos en todas las zonas sin tocar los vigentes. `PermissionCatalog` (`auth-service`) —el mapa rol-a-permisos que se embebe como *claim* en el *access token* para que `ticket-service/api-gateway` autoricen sin volver a consultar `auth-service` en cada petición— tampoco tenía prueba; un permiso mal listado aquí se propagaría en silencio a cada token emitido. Sus 4 pruebas nuevas incluyen una que compara los tres roles entre sí, no solo contra el valor esperado, para atrapar el caso de que dos roles compartieran por error la misma entrada del mapa.

Dos piezas más para cerrar esta ronda. `RefreshTokenMapper` (`auth-service`) —el mismo tipo de mapeo campo a campo entre dominio puro y entidad JPA que ya causó un bug real en `ticket-service` (ver `TicketMapper`, Entregable 10)— tampoco tenía prueba, con un riesgo de seguridad concreto y no solo de datos en blanco: `revoked` es el único mecanismo que impide reusar un *refresh token* ya rotado o robado (ver `TokenReuseDetectedException`); si este campo se perdiera en el mapeo, un token revocado se recargaría desde la base como si nunca lo hubiera sido. Sus 5 pruebas nuevas incluyen un caso dedicado a ese escenario, no solo el viaje de ida y vuelta genérico. Las funciones de mapeo de `apps/mobile` —`colorForStatus/labelForStatus/colorForPriority/labelForCategory` (`ui/theme/StatusColors.kt`)— tampoco tenían prueba: son *when* exhaustivos (el compilador de Kotlin exige cubrir todos los casos, así que nunca falta una rama), pero el compilador no verifica que el *valor* de cada rama sea el correcto —un color copiado por error de un estado a otro, o una tilde mal escrita en una etiqueta, pasarían el *build* sin que nada lo note. Sus 8 pruebas nuevas confirman, entre otras cosas, que ningún par de estados comparte color, y que **CRITICO** reutiliza a propósito el mismo tono de **ESCALADO** (documentado explícitamente en el comentario de la clase), no por accidente.

`AuthService` (`auth-service`) ya tenía una suite propia de 10 pruebas —incluida la detección de reuso de *refresh token* con revocación masiva—, pero con seis ramas reales sin ejercitar. Ninguna prueba cubría un correo ya registrado (`DuplicateEmailException`) ni un login contra un correo que directamente no existe —distinto del caso ya probado de contraseña incorrecta—, ambos debiendo fallar con el mismo `InvalidCredentialsException` genérico para no revelar si un correo está registrado. Tampoco había prueba de *refresh* con un *hash* de token desconocido ni con uno vencido pero no revocado —dos causas de fallo distintas del reuso ya cubierto—, ni de *logout* en absoluto: ni la revocación de una sesión activa, ni su idempotencia (cerrar sesión dos veces, o después de un *refresh* que ya revocó el mismo token, no debe volver a escribir en la base), ni el caso de un *token* desconocido. Tampoco `validate()` —el único método que traduce los *claims* ya verificados

del JWT a la respuesta que consume el *API Gateway*— tenía prueba propia. Sus 7 pruebas nuevas cierran los seis huecos.

TicketQueryService (svc-principal) —hallado revisando el reporte de cobertura JaCoCo del módulo, no por inspección manual— ya tenía una suite de 9 pruebas, pero cubría menos de la mitad de las ramas reales de la matriz de autorización por rol (**TicketAuthorization**, ver su tabla **CLIENTE/TECNICO/ADMIN**). De **getTicket()** solo estaba probado el camino *prohibido* de **TECNICO** (otra zona) y el de **ADMIN** contra un ticket inexistente —nunca el camino permitido de ninguno de los dos roles con un ticket real. De **listTickets()** faltaban las tres ramas con filtro de estado combinado con rol (**CLIENTE**, **TECNICO** y **ADMIN** con **zone+status**), la rama de **ADMIN** con solo zona, y —la más importante— el *fallback* final para un rol no reconocido: sin prueba, nada garantizaba que un rol nuevo o mal escrito cayera a lista vacía (*fail-closed*) en vez de, por accidente de refactor, colarse en alguna rama y filtrar tickets que no le correspondían. Sus 7 pruebas nuevas cierran los siete huecos, incluida esa última como caso explícito.

Dos manejadores de comando más, encontrados leyendo directamente el reporte JaCoCo del módulo (92.2 % y 95.1 % de cobertura respectivamente, no un 0 % como los hallazgos de rondas anteriores). **AssignTechnicianHandler** tenía solo 2 pruebas —el camino prohibido de **TECNICO** en otra zona, y la publicación del evento con **ADMIN**— sin cubrir ni el ticket inexistente, ni **CLIENTE** (siempre prohibido), ni el camino *permitido* de **TECNICO** en su propia zona, ni **TECNICO** sin zona reconocible. Sus 4 pruebas nuevas cierran esos cuatro huecos. **UpdateTicketStatusHandler** solo probaba **slaBreached** en su rama verdadera (resuelto después del plazo); la rama negativa —resuelto a tiempo— y el cortocircuito por **slaDeadline == null** (un ticket sin plazo formal, por ejemplo antes de que **ai-service** lo clasifique) nunca se ejercitaban —sin este segundo caso, quitar el chequeo de nulidad del **&&** habría tumbado con un **NullPointerException** el cierre de cualquier ticket sin SLA asignado, sin que ninguna prueba lo notara. Sus 2 pruebas nuevas cierran ambas ramas.

Tres huecos más, encontrados leyendo línea a línea el reporte HTML de JaCoCo (no solo el porcentaje agregado por clase). **UpdateTicketStatusHandler** quedaba en 95.1 % incluso después de la ronda anterior: la única línea marcada parcial era el **orElseThrow** del ticket inexistente —ninguna prueba llamaba al manejador con un **id** que el repositorio no encontrara. Su prueba nueva cierra esa última línea. **TicketClassificationListener** —el lado receptor de la Saga por coreografía, que aplica la clasificación de **ai-service** a un ticket— tenía tres pruebas, pero ninguna ejercitaba **parseEnumOrNull()** más allá del caso feliz: ni un valor de categoría desconocido (el escenario que el propio *javadoc* del evento anticipa: *ai-service y ticket-service pueden desincronizar su vocabulario*), ni un payload que sencillamente no trae categoría ni prioridad. Sus 2 pruebas nuevas confirman que ambos casos se ignoran en silencio —la categoría queda **null**, el ticket se guarda igual— sin tumbar el *listener*. **TelemetryGrpcClientAdapter** ya tenía una prueba de regresión de un bug real (el plazo de gRPC fijado una sola vez en el constructor en vez de por llamada), pero nunca ejercitaba su propio bloque **catch** —el comportamiento *fail-open* exigido por el ADR-0004: si **telemetry-service** está caído, se trata como "sin evidencia", nunca se propaga la excepción. Su prueba nueva apunta el adaptador a un puerto sin ningún servidor escuchando y confirma que devuelve **false** en vez de lanzar.

ThemeContext (apps/web) ya tenía dos pruebas, pero ambas montaban el *provider* sin nada guardado en **localStorage**, así que siempre ejercitaban el *fallback* a la preferencia del sistema operativo (**matchMedia**, mockeada en **test/setup.ts** sin preferencia oscura) —la rama real de usuario que vuelve", con un tema ya elegido en una visita anterior, nunca se probaba. Tampoco se confirmaba el propósito mismo de guardar el tema en **localStorage**: la prueba original de **toggle** solo verificaba el atributo **data-theme** del DOM, nunca que el nuevo valor quedara persistido. Sus 2 pruebas nuevas cierran ambos huecos: una guarda **"dark"** en **localStorage** antes de montar y confirma que el *provider* lo lee sin consultar **matchMedia**, la otra confirma que **toggle** deja el nuevo valor escrito

en `localStorage`, no solo reflejado en el DOM.

`CreateTicketModal` (`apps/web`, el único formulario que puede ver un `CLIENTE`) ya tenía cuatro pruebas, pero desproporcionadamente pocas para sus 155 líneas: cubrían el caso feliz, la validación con campos literalmente vacíos, y el error genérico de *fallo* de red, pero no el chequeo real —`title.trim()`, más estricto que `!title`— con campos de puro espacio en blanco; tampoco se probaba el camino donde teléfono/dirección sí se completan (recortados antes de enviarse, no solo el caso donde quedan `undefined`), ni que cambiar la zona del *select* realmente cambie la zona enviada —las cuatro pruebas existentes dejaban siempre la zona por defecto—, ni la guardia de cierre (clic en el fondo cierra, clic dentro del formulario no), el mismo patrón ya protegido en `TicketDetailModal` (Ronda 6) pero nunca replicado aquí. Sus 5 pruebas nuevas cierran los cinco huecos.

`TicketRowActions` (`apps/web`) ya tenía seis pruebas razonablemente completas —estado, auto-asignación de `TECNICO`, selector filtrado por zona de `ADMIN`, aviso sin técnicos— pero con dos huecos reales. Ninguna probaba el *guard* `if (status === ticket.status) return`: sin él, reelegir en el *select* el mismo estado que el ticket ya tiene dispararía un `PATCH` redundante cada vez. Tampoco se probaba, con rol `CLIENTE` (o sin rol), que el componente no ofreciera ningún control de asignación —defensa en profundidad más allá de que `ConsolePage` ya lo oculte para ese rol— ya que las seis pruebas existentes solo cubrían `TECNICO` y `ADMIN`. Sus 2 pruebas nuevas cierran ambos huecos.

`Badges` (`apps/web`) ya tenía cinco pruebas, pero con el mismo tipo de hueco que `StatusColors.kt` en móvil (Ronda 15): `STATUS_VAR` es un mapa manual estado-a-color —a diferencia del *when* exhaustivo de Kotlin, aquí ni siquiera el compilador obliga a cubrir los seis casos—, y las pruebas existentes solo verificaban la etiqueta traducida de *un* estado y *una* prioridad, nunca el color en sí. Un color copiado por error de un estado a otro habría pasado el *build* sin que nada lo notara. `AssignedChip` —reemplaza el UUID crudo del técnico por un chip genérico cuando lo ve un `CLIENTE`, simétrico a `UnassignedChip`, que sí tenía prueba— no tenía ninguna en absoluto. Sus 4 pruebas nuevas confirman que los seis estados tienen colores de fondo distintos entre sí, que las cuatro prioridades tienen colores de borde distintos entre sí, que `CRITICO` reutiliza a propósito el mismo tono que `ESCALADO` (documentado en el código, no accidental), y cubren `AssignedChip` por primera vez.

`useTickets` (`apps/web`, el equivalente de consola a `TicketListViewModel` en móvil, Ronda 5) ya tenía cinco pruebas, pero con cuatro ramas reales sin ejercitar. La búsqueda también filtra por `ticketId` además de descripción —documentado explícitamente en el propio código: sirve para verificar que una acción hecha desde otro cliente (la app móvil) se propagó al mismo backend—, pero solo se probaba la búsqueda por descripción. Una búsqueda de puro espacio en blanco nunca se probaba que se tratara como vacía. Las cuatro pruebas de filtro existentes probaban búsqueda, zona y estado por separado, nunca combinados —donde un `OR` por error en vez de `AND` pasaría desapercibido—. Y `refresh()` nunca se ejercitaba: ninguna prueba confirmaba que realmente volviera a pedir los tickets al backend. Sus 4 pruebas nuevas cierran los cuatro huecos.

`AdminPage` (`apps/web`, alta de cuentas) ya tenía cinco pruebas, pero con cuatro huecos reales para sus 215 líneas. Solo se probaba la creación exitosa dejando el rol por defecto (`CLIENTE`, con `zone: null`); el camino real donde el rol elegido es `TECNICO` —que exige una zona válida en el backend, ver `AuthService.validateZoneForRole`— nunca enviaba la zona seleccionada en una prueba. El único error de formulario probado era la validación local (campos vacíos); un fallo real de la API (correo duplicado, por ejemplo) nunca se ejercitaba. Tampoco se confirmaba que el formulario se limpiara después de crear una cuenta con éxito —solo el mensaje de éxito—, dejando abierta la posibilidad de que datos de la cuenta anterior quedaran arrastrados a la siguiente. Y, igual que en `CreateTicketModal` (Ronda 21), la validación real es `.trim()`, más estricta que la única prueba existente de campos literalmente vacíos. Sus 4 pruebas nuevas cierran los cuatro huecos.

`ReportsPage` (`apps/web`, lado de lectura del CQRS) tenía solo dos pruebas para sus 98 líneas:

el caso feliz con datos en los tres desgloses y el error de carga. Ningún desglose vacío se probaba —`BreakdownCard` renderiza un guion en ese caso, rama que nunca se ejercitaba—, tampoco el texto de carga antes de que la promesa resolviera, ni el cálculo real del ancho de barra (`value/max*100%`, con `max = Math.max(1, ...)` para evitar dividir entre cero). El hallazgo más interesante: `ReportsPage` mantiene su *propia* copia de `STATUS_VAR`, independiente de la de `Badges.tsx` (Ronda 23) —dos mapas manuales que deben mantenerse sincronizados a mano, sin nada que los ate—, y su `colorFor(key) ?? 'var(-color-navy)'` para un estado no reconocido nunca se probaba. Sus 4 pruebas nuevas cierran los cuatro huecos.

`LoginPage` (`apps/web`) tenía solo dos pruebas para sus 281 líneas —la mayoría constantes de estilo, pero con lógica real de todas formas—: la validación local de campos vacíos y que escribir la limpia. Ninguna de las dos ejercitaba `authApi.login` de verdad —quedaba sin mockear, así que cualquier intento de *submit* real habría fallado por red antes de poder observarse—, así que el flujo completo de autenticación nunca se probaba de punta a punta: ni que un *login* exitoso navegue a `/main`, ni que el correo se recorte antes de enviarse (la contraseña, deliberadamente, no), ni que un fallo del backend muestre el mensaje de error real en vez de quedar mudo. Tampoco se probaba el botón del ojo que alterna el campo de contraseña entre oculto y visible. Sus 4 pruebas nuevas —mockeando `authApi.login` con el mismo criterio que `useAuth.test.tsx`— cierran los cuatro huecos.

`AppLayout` (`apps/web`, cabecera y navegación de toda página autenticada) ya tenía cuatro pruebas, pero con tres huecos reales. `CLIENTE` —el rol más común del sistema— nunca se probaba de forma explícita para la regla de RBAC que oculta `Administración/Reportes`: solo se cubrían "sin sesión" `ADMIN`. El botón `Salir` solo se confirmaba cerrando la sesión (`isLoggedIn() === false`); que también navegara a `/login` nunca se verificaba de punta a punta. Y, cuarta aparición del mismo patrón de hueco (`Badges.tsx`, Ronda 23; `ReportsPage`, Ronda 26; `StatusColors.kt` en móvil, Ronda 15): `ROLE_BADGE_COLOR` es otro mapa manual rol-a-color, y las pruebas existentes solo verificaban la etiqueta traducida (`"administrador"`, `"Técnico"`), nunca que los tres roles tuvieran colores de insignia distintos entre sí. Sus 3 pruebas nuevas cierran los tres huecos.

`SettingsPage` (`apps/web`) ya tenía dos pruebas —cambiar idioma, cambiar tema—, pero ninguna cubría el *guard mode* `!=='light' && toggle()`: sin él, reclicar el botón del tema ya activo llamaría a `toggle()` igual y cambiaría al otro tema por error, el mismo tipo de *guard* redundante ya protegido en `TicketRowActions` (Ronda 22) para el estado del ticket. Tampoco se comparaba `pillStyle(active)` entre el botón activo y el inactivo —quinta aparición del patrón de hueco de color/estilo nunca verificado (`Badges.tsx`, Ronda 23; `ReportsPage`, Ronda 26; `StatusColors.kt` en móvil, Ronda 15; `AppLayout`, Ronda 28)—. Sus 2 pruebas nuevas cierran ambos huecos.

Se cambió de capa para esta ronda: dos traductores centrales de excepciones a respuestas HTTP (`@RestControllerAdvice`) sin ninguna prueba, a diferencia de su equivalente ya cubierto en `svc-principal`. El de `auth-service` traduce seis excepciones distintas —cada una a su código de estado correcto (404/400/401/403/500), incluida la única lógica real del archivo: unir varios errores de campo de validación con `" ; "` o caer a un mensaje genérico cuando no hay ninguno—; sus 8 pruebas nuevas cubren los seis *handlers*, ambas ramas de la unión de errores, y confirman explícitamente que `handleAccessDenied` usa un mensaje propio fijo en vez de reenviar `ex.getMessage()` —si alguien lo cambiara por error, podría filtrar detalles internos de Spring Security en la respuesta—. El de `report-service` es más simple (`ForbiddenException` → 403, cualquier otra cosa → 500); sus 2 pruebas nuevas lo cubren por completo.

`TicketRepositoryAdapter` era, según el reporte JaCoCo, la clase peor cubierta de todo el módulo (29.2%): no tenía ninguna prueba unitaria propia, solo se ejercitaba de forma indirecta a través de `TicketRepositoryIntegrationTest` (Testcontainers, un CockroachDB real), que no corre localmente en Windows por la limitación de *Docker outside of Docker* ya documentada en la Sección de

amenazas —así que en la práctica, fuera del *runner* de GitHub Actions, esta clase no tenía cobertura real alguna. Sus 8 pruebas nuevas cubren los ocho métodos del adaptador (siete consultas más `save()`), usando el `TicketMapper` real, no mockeado, para que un cambio accidental en el mapeo también rompa esta prueba. `IncidenciaRepositoryAdapter`, su equivalente para incidencias, ya tenía cinco pruebas pero dejaba sin cubrir `findByZone()`; su prueba nueva cierra ese último método. `TicketRepositoryAdapter` queda en 100 % de cobertura de línea según el propio reporte JaCoCo tras esta ronda.

`logger.js` (`notification-service`) no tenía ninguna prueba: es la implementación propia del logging estructurado en JSON exigido por la Práctica Experimental U5 (ítem 2a, "*logging estructurado (JSON) en todos los microservicios*") —deliberadamente sin dependencia nueva (nada de `winston/pino`), así que ninguna librería externa ya probada por terceros garantiza que la línea impresa sea JSON válido con los campos correctos; esa garantía solo la puede dar una prueba propia. Sus 4 pruebas nuevas confirman que `info()/error()` escriben en la salida correcta (`console.log` vs. `console.error`, la única rama real del archivo), que los campos `extra` se mezclan en la línea, y que sin ellos no queda un `undefined` colado en el JSON resultante.

12.1.1. Contrato de interfaz (OpenAPI)

Antes de que exista una sola prueba de contrato hace falta el contrato en sí. El archivo `docs/api/openapi.yaml` (OpenAPI 3.0.3) describe, de forma verificable por herramienta y no solo en prosa, las diecisiete operaciones REST de los cinco microservicios: registro, login, refresco, logout, validación de sesión y administración de cuentas (`auth-service`, siete operaciones); creación, listado, detalle, cambio de estado y asignación de tickets (`ticket-service`, cinco); notificaciones (`notification-service`, una); clasificación automática (`ai-service`, una); y los tres puntos de agregación de `report-service`, incluida la exportación a CSV. Cada una de las diecisiete declara su esquema de respuesta exitosa, los códigos de estado posibles, y el esquema de seguridad (`bearerAuth`, JWT) cuando aplica; se valida automáticamente en el flujo con Spectral (Sección 12.1.2), que rompe la construcción si alguna deja de cumplirlo.

Este archivo no es un anexo estático: es la fuente que el contrato Pact de la siguiente subsección referencia como forma esperada, y es lo que permite a un integrante nuevo entender la API completa sin leer el código de los cinco servicios. Se actualizó para esta entrega —seguía describiendo los puertos directos de la Entrega 2 como si fueran la única vía de acceso, sin mencionar el API Gateway que desde la Entrega 4 es el punto de entrada real de ambos clientes (Sección 9)— para reflejar las dos rutas reales: el *gateway* en el puerto 8000 como camino de producción, y los puertos directos de cada servicio conservados solo como vía de depuración local.

12.1.2. Validación automática del contrato

Que el contrato declare un esquema no basta si nada impide que una operación nueva se añada sin uno: la Entrega 4 se calificó con 4 operaciones sin esquema de respuesta señaladas por el profesor, y al revisar las diecisiete resultaron ser once, no cuatro. Para que ese hueco no vuelva a pasar desapercibido, el trabajo `validate-openapi` del flujo (`.github/workflows/ci-cd.yml`) corre Spectral sobre `docs/api/openapi.yaml` en cada envío y cada *pull request*, con una regla propia declarada en `.spectral.yaml` (`response-schema-required`) que exige `content.application/json.schema` en toda respuesta 2xx; si una operación futura lo omite, la construcción falla con el nombre exacto de la operación incumplida, no con una advertencia que se pueda ignorar.

12.1.3. Pruebas de contrato

Se adoptó *consumer-driven contract testing* con Pact entre **apps/web** (consumidor) y **ticket-service** (proveedor). El consumidor genera un archivo de contrato (**pacts/soporte-web-ticket-service.json**) a partir de las expectativas reales del cliente sobre la forma de la respuesta; el proveedor lo verifica reproduciendo cada interacción contra el servicio real. Una particularidad de esta arquitectura obligó a una decisión de diseño: como **ticket-service** valida cada JWT llamando por HTTP a **auth-service** (**AuthGatewayFilter**) en vez de verificar la firma localmente, la verificación del proveedor no puede correr en un contexto de prueba aislado —necesita el *stack* completo levantado y un JWT real obtenido de un inicio de sesión real.

Un segundo ajuste, encontrado al ejecutar la verificación por primera vez, fue de modelado del contrato: los campos **technicianId** y **resolvedAt** son legítimamente nulos o no nulos según el estado real del ticket, y un solo ejemplo con matiz de tipo (**like(null)**) no puede expresar ambos casos a la vez sobre el mismo campo. Se adoptó el patrón oficial de Pact para colecciones con formas mixtas (**arrayContaining** con dos variantes representativas), en vez de forzar una condición más débil o más fuerte que la que el sistema realmente cumple.

12.2. Pipeline de CI/CD

El archivo **.github/workflows/ci-cd.yml** define diez *jobs* en un grafo dirigido acíclico, siguiendo la cultura DevOps de que ningún cambio llegue a un artefacto publicado sin pasar por los mismos controles [32, 31]:

Tabla 15: Los diez jobs del pipeline.

#	Job	Qué hace
1	lint	ESLint (web) + Android Lint (móvil)
2	validate-openapi	Spectral valida docs/api/openapi.yaml contra la regla propia response-schema-required
3	test-backend	Los cinco servicios Java con pruebas automatizadas + Node + Python, pruebas unitarias e integración
4	test-web	Vitest, incluido el contrato Pact del consumidor
5	test-mobile	Unitarias JVM + instrumentadas en un emulador Android headless real
6	build-images	Construye y publica las ocho imágenes Docker en GHCR (solo en <i>push</i>)
7	build-mobile-apk	Compila, firma y publica el APK de <i>release</i> junto al manuscrito, bajo una etiqueta de versión que se mueve al commit actual en cada publicación
8	integration	Levanta el <i>stack</i> completo y corre Pact proveedor + Playwright contra el backend real
9	compile-latex	Recompila el manuscrito y falla ante referencias/citas indefinidas o etiquetas duplicadas
10	verify-checksums	Verifica los tres manifiestos SHA256SUMS versionados contra los datos crudos

Los *jobs* 6 y 7 declaran **needs** sobre los *jobs* de prueba correspondientes: GitHub Actions no los ejecuta hasta que sus dependencias terminen en verde. Esto no siempre fue así de completo: en una revisión externa del pipeline se encontró que **build-images** solo dependía de **test-backend** y **test-web**, así que publicaba imágenes en GHCR aunque **lint** o **test-mobile** estuvieran en rojo –

no era la compuerta de calidad real que se pretendía. Se corrigió agregando esas dos dependencias faltantes. Una segunda revisión encontró que aún faltaba la más importante: `integration` – el único *job* que prueba contra el *stack* real y no contra *mocks* – corría en *paralelo* con `build-images` en vez de antes, así que una imagen podía publicarse aunque la integración real fallara. Se agregó también a `needs`, sin crear una dependencia circular porque `integration` construye sus propias imágenes en local (`docker compose up -build`), no depende de las que publica GHCR. La declaración final de `build-images` es `needs: [lint, test-backend, test-web, test-mobile, integration]`.

Una tercera revisión, esta vez sobre `build-mobile-apk` —el *job* que firma el APK y publica el *release* de la entrega—, encontró la misma clase de hueco: su declaración original era `needs: [test-mobile, compile-latex]` únicamente, así que el instalable firmado y el manuscrito se publicaban aunque `lint`, `validate-openapi` o la suite completa de `test-backend/test-web/integration` estuvieran en rojo. Se corrigió a `needs: [lint, validate-openapi, test-backend, test-web, test-mobile, integration, compile-latex]`, para que tampoco este artefacto se publique sin que los siete jobs de verificación pasen primero.

Un segundo defecto de `build-mobile-apk`, independiente del anterior: la etiqueta bajo la que se publicaba (`mobile-release`) no era una etiqueta de *versión* en ningún sentido real. `softprops/action-gh-release` reutiliza el *ref* de git de un tag ya existente en vez de moverlo al commit actual, así que aunque los binarios adjuntos se reconstruían en cada *push* a `main`, el tag de git seguía apuntando al commit donde se creó por primera vez —etiqueta y contenido publicado dejaban de corresponder silenciosamente. Se agregó un paso explícito (`git tag -f ... && git push -force`) que mueve la etiqueta `v1.0-entrega-final` al commit actual *antes* de publicar, y se renombró la etiqueta en consecuencia: ahora es una versión real, no un alias fijo que solo indica "la más reciente".

12.3. Depuración real del job integration

Los primeros 6 *jobs* pasaban de forma consistente desde su introducción. El séptimo, `integration`, falló en *todas* las corridas sobre GitHub Actions durante varios días, pese a que el mismo `docker compose up -d -build` que ese *job* ejecuta funcionaba sin problema en las máquinas de desarrollo del equipo. Esa asimetría – funciona en local, falla siempre en el *runner* – fue la pista de que la causa era estructural, no una falla puntual. El equipo instaló la CLI oficial de GitHub (`gh`) para acceder al log real de cada paso (la API pública solo devuelve “`Process completed with exit code 1`”, sin el comando que realmente falló), y diagnosticó tres causas raíz independientes, cada una confirmada con evidencia real antes de corregirla:

1. **Candado circular en el healthcheck de CockroachDB.** El healthcheck de los 3 nodos usaba `/health?ready=1` – que solo responde OK una vez que el nodo ya pertenece a un clúster *inicializado* – mientras que `db-init` (quien ejecuta `cockroach init`) esperaba a que ese mismo healthcheck pasara primero. Nadie quedaba listo porque nadie los inicializaba, y nadie los inicializaba porque nadie quedaba listo. En desarrollo esto no se manifestaba porque los volúmenes de Docker ya traían un clúster inicializado de sesiones anteriores; un *runner* de GitHub Actions arranca siempre desde un volumen vacío, así que el candado se activaba sin excepción. Un primer intento de aumentar el presupuesto de tiempo del healthcheck *no* resolvió el problema – solo retrasó la falla de 70 s a 3 min 21 s –, confirmando que la causa era estructural y no de tiempo. Se corrigió cambiando el healthcheck a `/health` (solo *liveness*) y agregando reintentos reales alrededor de `cockroach init`.
2. **Datos de demo del contrato Pact, nunca versionados.** El método `@State` de la verificación del proveedor asumía datos de demo ya cargados por un script que resultó no existir en ningún commit del repositorio – solo había existido, sin documentarlo como tal, en el volumen de Docker de quien lo escribió originalmente. Se corrigió haciendo que el propio `@State` inserte

esos datos por SQL directo, que es precisamente para lo que existe ese mecanismo de Pact.

3. **Playwright solo instalaba Chromium.** `playwright.config.ts` define 3 proyectos (Chromium, Firefox, WebKit – requisito de compatibilidad de esta entrega), pero el paso de instalación del *workflow* tenía el argumento `chromium` agregado, restringiendo la instalación a un solo navegador. Se corrigió quitando el argumento.

Las tres correcciones se verificaron de punta a punta contra el *stack* real – nunca simuladas – antes de subirlas, y el pipeline completo quedó en verde sobre un mismo commit. El patrón común a los tres problemas es el mismo: algo que *parecía* funcionar porque la máquina de desarrollo conservaba un estado acumulado (un volumen ya inicializado, datos cargados a mano, un navegador ya instalado) que un entorno realmente limpio nunca reproduce por su cuenta. El detalle completo de cada diagnóstico, incluidos los registros reales y los intentos que no funcionaron, está en `docs/ci_cd/informe_ci_cd.tex`.

12.4. Cobertura y complejidad ciclomática

La cobertura de `ticket-service`, medida con JaCoCo, no se resume en una sola cifra: la Tabla 16 reporta las tres métricas que la propia herramienta entrega, cada una nombrada, en vez de citar solo la más favorable (líneas) sin decir que es la de líneas.

Tabla 16: Cobertura de `ticket-service`, las tres métricas de JaCoCo

Métrica	Cubierto	Total	%
Líneas	692	716	96.6 %
Instrucciones	3392	3813	89.0 %
Ramas	127	162	78.4 %

Las tres cifras son ciertas simultáneamente y miden cosas distintas: líneas cuenta sentencias ejecutadas, instrucciones cuenta a nivel de bytecode (más granular, penaliza más una rama sin cubrir dentro de una misma línea), y ramas cuenta específicamente las bifurcaciones condicionales (`if/switch`/operadores lógicos de corto circuito) — es la más exigente y la que ya superó ampliamente el objetivo del 70 % que el proyecto se propuso desde la Entrega 3 (78.4 %, tras las rondas de pruebas de esta entrega). El paquete `infrastructure.security` (filtro y configuración de JWT), que en una revisión externa anterior a esta corrección tenía **0 de 38 líneas cubiertas** —ningún test ejercitaba las ramas de token inválido/expirado/ausente directamente sobre esas clases, solo indirectamente vía las pruebas de integración HTTP 401/403, que JaCoCo no le atribuye—, quedó cerrado con `AuthGatewayFilterTest` (38 de 38 líneas cubiertas ahora). El paquete que más arrastra la cifra de ramas hacia abajo hoy es `domain`, y en concreto una sola clase: `TicketId` (0 de 22 ramas cubiertas). No es una omisión: `TicketId` es la clave compuesta `@IdClass` de `Ticket` (`createdAt`, `id`), generada por Lombok (`@Data`) para `equals()`/`hashCode()`; igual que el código generado por `protobuf` más abajo en esta sección, ninguna convención de pruebas exige ejercitar a mano cada rama de un `equals` mecánico sobre dos campos, y el propio JPA nunca invoca esa clave compuesta en tiempo de ejecución —el punto de acceso real es `TicketRepository.findById(UUID)`, como documenta el comentario de la clase—, así que se declara aquí como brecha conocida y deliberadamente no priorizada, no oculta.

El reporte XML de JaCoCo agrega, a nivel de `<report>`, un total de 687/711 líneas —no las 692/716 que resultan de sumar el CSV fila por fila. La diferencia no es un error de cálculo: cinco archivos fuente (`AuthGatewayFilter.java`, `Incidencia.java`, `Ticket.java`, `IncidenciaJpaEntity.java` y `TicketJpaEntity.java`) contienen dos clases cada uno —una externa y una interna

o lambda generada por el compilador—, y el CSV reporta la cobertura por clase, de modo que las líneas físicas compartidas entre ambas clases del mismo archivo se cuentan dos veces al sumarlas. El XML, en cambio, agrega por archivo fuente y cuenta cada línea física una sola vez. Las cifras de instrucciones y ramas no sufren este efecto.

Ninguna de las tres cifras incluye el código generado automáticamente por `protobuf/ grpc-java` a partir de `telemetry.proto` (*getters*, *setters* y constructores mecánicos sin lógica propia, que ninguna convención de pruebas exige cubrir a mano) — se excluye explícitamente en el `pom.xml`, con un comentario que documenta por qué. Sin esa exclusión, ese código generado por sí solo diluía la cifra reportada de 59.3 % (antes de que `telemetry-service/CORREL` existieran) a un engañoso 42.7 %, contando en contra código que nadie prueba por convención en ningún proyecto Java real. El reporte HTML de JaCoCo que respalda estas tres cifras —no solo una tabla escrita a mano— está publicado en `docs/evidencias/jacoco-svc-principal/`, junto con el archivo crudo `jacoco.csv` del que se recalcularon directamente (no se re-tipearon a mano) y el binario crudo (`jacoco-svc-principal-raw.exec`) para quien quiera regenerarlo o verificarlo con otra herramienta.

Hasta esta entrega, el objetivo del 70 % del Módulo A no se hacía cumplir: JaCoCo vivía en un perfil Maven que no se activaba por defecto (`mvn test -Pcoverage`), el flujo de CI corría `mvn test` sin ese perfil, y la ejecución `check` —la que compara contra un umbral y puede romper la construcción— no existía en ningún sitio. El pipeline nunca medía nada, pese a que el propio comentario del `pom.xml` decía que el reporte «es usado por el pipeline». Se movió el complemento fuera del perfil (ahora siempre activo) y se agregó una ejecución `check` con un umbral del 70 % sobre el contador de instrucciones, ligada a la misma fase `test` que ya corre en `test-backend`: si la cobertura cae por debajo de ese umbral, `mvn test` falla con la razón exacta.

Antes de escribir pruebas nuevas para esta entrega, el código propio (sin el generado) ya estaba en 64.4 %. Se identificaron con el propio reporte de JaCoCo las clases reales con mayor número de líneas sin cubrir — no una elección arbitraria — y se agregaron 31 pruebas unitarias nuevas en esa ronda inicial (la cifra total del módulo, acumulada tras todas las rondas posteriores, es la que reporta la Tabla 14, no un número fijo en este párrafo) sobre, entre otras: `CorrelationService` (el orquestador de `CORREL`, que llegó a esta entrega prácticamente sin pruebas propias), los tres observadores concretos del patrón Observer (`EscalationLoggingObserver`, `EscalationMetricsObserver`, `EscalationEventPublisherObserver` — cerrando un hueco real en un patrón que el propio manuscrito documenta como implementado), `GlobalExceptionHandler`, y el mapeo/adaptador de persistencia de `Incidencia`.

La complejidad ciclomática promedio del mismo módulo, medida con la herramienta `lizard`, es de 1.8, muy por debajo del umbral de 10 — ninguna función individual supera siquiera el umbral de alarma de 15, resultado consistente con el refactor a métodos pequeños de la Sección 9.4.

13. Observabilidad distribuida

La observabilidad de un sistema distribuido se sustenta en tres señales complementarias: métricas, registros estructurados y trazas correlacionadas por un identificador común [33, 34]. El estándar OpenTelemetry [3] unifica su modelo de datos y su exportación, permitiendo que distintos proveedores de *backend* consuman las mismas señales sin instrumentación duplicada.

13.1. Métricas

Se reutilizan las métricas de CockroachDB ya expuestas desde la Entrega 3 (`crdb_query_duration_seconds`, `crdb_transaction_retries_total`, `crdb_pool_active_connections`) y se agre-

gan cuatro métricas nuevas de aplicación, recolectadas por Micrometer y expuestas en `/actuator/prometheus` de cada servicio Java (y su equivalente en `api-gateway` vía un *filter* reactivo):

- `http_requests_total` — contador de peticiones, etiquetado por ruta, método y código de estado.
- `http_request_duration_seconds` — histograma de latencia, mismas etiquetas.
- `app_active_sessions` — *gauge* de sesiones activas reales, respaldado por el conteo de *refresh tokens* no revocados y no expirados en `auth-service`.
- `app_business_events_total` — contador de eventos de negocio; el primer y único evento real que el sistema dispara por sí mismo es el escalado de un ticket (patrón Observer, Sección 9.4).

En las rutas de `api-gateway` y `ticket-service`, los identificadores de recurso (UUID) se normalizan a `{id}` en la etiqueta de ruta antes de registrar la métrica, para evitar una explosión de cardinalidad de series temporales distintas por cada ticket individual.

13.2. Registros estructurados

Los cuatro servicios Java emiten registros en JSON estructurado con Logback (`LogstashEncoder`), incluyendo el campo `trace_id`. El agente Java de OpenTelemetry, adjuntado en tiempo de ejecución vía `-javaagent`, escribe automáticamente `trace_id` y `span_id` en el contexto de diagnóstico (MDC) de cada hilo que procesa una petición instrumentada —la configuración de Logback solo necesita leer esas dos claves, sin ningún cambio en el código de la aplicación.

13.3. Trazas distribuidas

El mismo agente Java exporta trazas vía OTLP a un **OpenTelemetry Collector**, que las reenvía a **Grafana Tempo** para su almacenamiento y consulta. Una traza típica de una petición `GET /api/v1/tickets` atraviesa: el span de recepción en `api-gateway`, el span de la llamada HTTP saliente a `auth-service /validate`, el span del *handler* de aplicación en `ticket-service`, y el span de la consulta JDBC a CockroachDB —los cuatro correlacionados por el mismo `trace_id` que aparece en los registros de cada servicio involucrado.

Limitación conocida: la aplicación móvil no incluye un SDK de OpenTelemetry propio, así que la traza visible de una petición originada en el cliente móvil comienza en `api-gateway`, no dentro del proceso del teléfono. Se documenta esta limitación explícitamente en vez de presentar una instrumentación de cliente que no existe.

13.4. Dashboard operativo

El dashboard de Grafana, versionado como JSON reproducible en `ops/grafana/pfc-dashboard.json`, incluye los seis paneles obligatorios: tasa de peticiones, latencia p50/p95/p99, tasa de errores 5xx, disponibilidad Raft del cluster CockroachDB (nodos vivos, rangos no disponibles), uso de recursos por contenedor (vía cAdvisor, única forma práctica de obtener CPU/memoria por contenedor ya que los `/actuator/prometheus` de cada servicio solo reportan métricas de su propia JVM), y latencia extremo a extremo, proxied a través de las métricas de `api-gateway` por ser el único punto por el que pasan absolutamente todas las peticiones de ambos clientes.

14. Evaluación experimental ISO/IEC 25010

El modelo de calidad ISO/IEC 25010:2023 [1] organiza la calidad de producto en ocho características; esta entrega evalúa las cinco exigidas por la guía, cada una con al menos una métrica

cuantitativa y su umbral objetivo (Tabla 17), siguiendo el rigor de reporte experimental de Jain [20] y Wohlin et al. [19].

14.1. Preguntas de investigación

Siguiendo el criterio de Wohlin et al. [19] de que una pregunta de investigación debe admitir una respuesta negativa para ser una pregunta y no una afirmación disfrazada, esta entrega formula tres, cada una respondida con datos ya obtenidos y no con una expectativa:

1. **PI1.** ¿El sistema, bajo la carga concurrente medida, cumple los umbrales objetivo de eficiencia de desempeño ($p95 < 500$ ms) y fiabilidad (error $5xx < 1$ %) de ISO/IEC 25010? **Respuesta:** **no**, en ninguna de las dos —ver Sección 14 de este mismo documento (subsección *Eficiencia de desempeño y fiabilidad*), con causa raíz identificada y no solo el síntoma.
2. **PI2.** ¿El sistema mitiga las seis categorías de riesgo OWASP más aplicables a su arquitectura REST + JWT? **Respuesta:** **no completamente** —5 de 6, con una brecha real y concreta documentada (ausencia de límite de intentos de inicio de sesión), no cinco de seis presentadas como éxito total.
3. **PI3.** Antes de esta auditoría, ¿el pipeline de CI/CD garantizaba que ningún artefacto se publicara en el registro sin que las cinco verificaciones automatizadas pasaran primero? **Respuesta:** **no** —**build-images** solo dependía de dos de los cinco *jobs* de verificación, y corría en paralelo con el sexto (**integration**) en vez de después. Ver Sección 12 para el diagnóstico completo y la corrección aplicada.

Las tres preguntas podían haberse respondido “sí” y el resultado seguiría siendo válido y reportable: el objetivo no era producir un resultado favorable, sino uno verificable.

14.2. Nota sobre el alcance

Por restricción de tiempo de entrega, el equipo decidió correr una **escala reducida** del experimento de carga: **r=5 repeticiones** por escenario (en vez de las 10 exigidas), con corridas más cortas (60–120 s en vez de 5–10 min). La metodología de análisis —descartar la primera y la última repetición, calcular media e intervalo de confianza al 95 % sobre las intermedias— es la misma que exige el protocolo, aplicada a menos muestras. El detalle completo, con los datos crudos de las diez corridas reales, está en `docs/experimentos/evaluacion_iso25010.md`.

Tabla 17: Las cinco características evaluadas, con su umbral objetivo y el resultado medido.

Característica	Umbral	Medido	¿Cumple?
Eficiencia de desempeño	$p95 < 500$ ms	1.4–1.6 s bajo carga concurrente	No
Fiabilidad	Error $5xx < 1$ %	19–34 % bajo carga concurrente	No
Seguridad	100 % (acceso + OWASP)	30/30 control de acceso; 5/6 categorías OWASP mitigadas	Parcial
Mantenibilidad	Cobertura ≥ 70 % + CCN < 10	96.6 % cobertura; CCN medio 1.8	Sí
Compatibilidad	Pasa suite E2E	9/9 en 3 navegadores; API 26+ verificado por <i>lint</i>	Sí

14.3. Eficiencia de desempeño y fiabilidad

Se ejecutaron dos escenarios de carga con Locust contra el API Gateway: (A) 30 usuarios en carga plana durante 60 s, y (B) una rampa de 0 a 60 usuarios durante 120 s. En ambos, la tasa de error observada (19 % y 34 % respectivamente) superó ampliamente el umbral del 1 %. La causa raíz, confirmada directamente en los registros de `auth-service` y `ticket-service`, es un mensaje explícito del motor de base de datos:

```
ERROR: No license installed. The maximum number of concurrently open transactions
has been reached.
```

Es decir: bajo concurrencia real, **CockroachDB Core sin licencia** rechaza nuevas transacciones al superar su límite de transacciones concurrentes abiertas —una limitación documentada del propio motor en su edición gratuita, no un defecto de la lógica de reintento de la aplicación (`TicketWriter.saveWithRetry`, ya cubierta por pruebas unitarias). Con una licencia comercial, o migrando a CockroachDB Serverless, este límite no existiría.

14.4. Seguridad

Se verificó el control de acceso con 30 peticiones reales contra el API Gateway (10 por caso): sin encabezado `Authorization`, con un token de formato inválido, y con un usuario autenticado de rol incorrecto intentando un endpoint de administración —los tres casos fueron rechazados correctamente el 100 % de las veces (401/401/403).

Para la ausencia de vulnerabilidades OWASP [38], un escaneo automatizado completo (OWASP ZAP) quedó fuera del alcance de tiempo de esta entrega; en su lugar se verificaron directamente, por código y en vivo, las seis categorías más aplicables a esta arquitectura REST + JWT. Cinco están mitigadas (control de acceso, criptografía —contraseñas con `BCryptPasswordEncoder`, JWT firmado HS256—, inyección —consultas parametrizadas vía JPA, sin SQL nativo concatenado—, configuración de `actuator` —solo expone `health`, `info`, `prometheus`—, y registro/monitoreo). Se encontró una **brecha real**: no existe límite de intentos de inicio de sesión (*rate-limiting* o bloqueo de cuenta), por lo que un atacante puede reintentar contraseñas sin restricción.

14.5. Mantenibilidad

Ver Sección 12: cobertura de 96.6 % de líneas propias (por encima del objetivo del 70 %, sin contar código generado por protobuf/gRPC) y complejidad ciclomática media de 1.8 (muy por debajo del umbral de 10).

14.6. Compatibilidad

La misma suite Playwright de la Sección 12 se ejecutó contra tres motores de renderizado (Chromium, Firefox y WebKit —este último equivalente a Safari, que no corre de forma nativa en Windows/Linux). Las nueve pruebas pasan en los tres motores. Para la aplicación móvil, la evidencia disponible en este ciclo es el `minSdk=26` declarado en Gradle (una garantía de tiempo de compilación) y el resultado limpio de la regla `NewApi` de Android Lint, que falla si el código usa una API de plataforma por encima del mínimo sin una guarda de versión explícita.

14.7. Amenazas a la validez de este experimento

Internas: (1) la escala reducida (5 repeticiones, no 10) amplía los intervalos de confianza frente al protocolo completo; (2) Locust, el *stack* completo de 17 contenedores y el propio proceso de

verificación compitieron por CPU en una sola máquina de desarrollo, introduciendo ruido en la latencia medida; (3) el límite de licencia de CockroachDB es una propiedad de este entorno de demostración, no necesariamente de un despliegue con licencia. **Externas:** (1) los datos usados son sintéticos, no tráfico real de un ISP; (2) dos de los diez tipos de prueba de la pirámide completa (las instrumentadas de Android) no se ejecutaron en este ciclo por falta de un emulador conectado en el momento de la medición.

15. Resultado del experimento CORREL

Esta sección responde la pregunta que da sentido a la Adición 1 de la Ampliación del Módulo G (Sección 9.6, docs/adr/0008-correl-incidencias.md): ¿agrupar tickets por zona, ventana deslizante y telemetría (c1/c2) mejora realmente el agrupamiento de una avería frente a no correlacionar (c0), y a qué costo en falsos positivos? El procedimiento completo, el entorno congelado y los datos crudos están en experimentos/protocolo-e4.md y experimentos/resultados/; esta sección resume el análisis.

15.1. Escala ejecutada

Por restricción de tiempo se ejecutó una escala reducida frente al protocolo completo de la Sección 5.4 de la guía de reutilización (10 repeticiones, 100–500 abonados por escenario): **5 repeticiones** por condición, con **20 abonados** (Esc-2, avería masiva), **60 abonados** (Esc-3, avería mayor) y **20+20 abonados en dos zonas** (Esc-4, dos averías simultáneas), bajo las tres configuraciones c0/c1/c2 sin reducir. Esto da 45 corridas reales (60 filas de resultado, ya que Esc-4 reporta una fila por zona) contra el stack completo levantado con `docker compose up -d -build`, no simuladas. El Esc-1 (carga nominal sin avería) no se repite aquí: es el mismo escenario A de resultados/locust/ ya usado para la Tabla 17. Entre cada corrida se vació el estado de correlación (`TRUNCATE incidencias, incidencia_tickets`) para eliminar contaminación entre repeticiones, ya que el servicio busca candidatas por zona y ventana de 15 minutos sin distinguir de qué corrida provienen.

15.2. Resultado por configuración

Tabla 18: Precisión y exhaustividad del agrupamiento por configuración CORREL (n=20 por configuración)

Configuración	Precisión	Exhaustividad	Incidencias efectivas (media)
c0 (sin correlación)	1.00	0.04	30.0
c1 (zona+ventana)	1.00	1.00	1.0
c2 (zona+ventana+telemetría)	1.00	1.00	1.0

La precisión es 1.00 en las tres configuraciones porque, dado el diseño de una sola avería por zona y ventana, ningún ticket ajeno a la avería inyectada entra jamás a la incidencia que se compara —no hay falsos positivos que penalicen la precisión con este diseño de escenario—. La diferencia real está en la exhaustividad: c0 abre una incidencia por ticket (línea base, exhaustividad $\approx 1/N$ por diseño: cada ticket es su propia incidencia), mientras que c1 y c2 agrupan el 100 % de los tickets de la avería en una sola incidencia, reduciendo la carga real de trabajo del despachador de N incidencias a 1 por avería.

15.3. Contraste estadístico

Con Mann–Whitney U (bilateral) sobre la exhaustividad de **c2** contra **c0** ($n=20$ por grupo): $U = 400$, $p \approx 2,58 \times 10^{-9}$, con tamaño del efecto de Vargha–Delaney $\hat{A}_{12} = 1,0$ —separación completa: en las 20 corridas de **c2**, la exhaustividad observada superó a las 20 de **c0** sin una sola excepción—. **Se rechaza H0**: la diferencia entre correlacionar y no correlacionar es estadísticamente significativa y de magnitud máxima bajo esta métrica y esta escala.

15.4. Contraste **c1** vs **c2**: ¿aporta algo la telemetría?

La pregunta de investigación de esta sección compara tres configuraciones, no dos. El contraste anterior (**c2** contra **c0**) responde si correlacionar ayuda, pero no responde la pregunta que en realidad justifica construir **c2** sobre **c1**: ¿la telemetría de PE-U1 (Sección 9.6) aporta algo que la sola coincidencia de zona y ventana (**c1**) no aporte ya?

Comparando **c1** contra **c2** con Mann–Whitney U sobre las 20 corridas de cada una (precisión y exhaustividad) y sobre las incidencias efectivas: **los tres valores son idénticos, corrida por corrida**, en las 20 repeticiones. No hay una sola corrida donde **c1** y **c2** difieran. El tamaño del efecto de Vargha–Delaney es exactamente $\hat{A}_{12} = 0,5$ (indistinguible al azar) porque no hay diferencia que medir.

Esto es un resultado real, y hay que decirlo con la misma franqueza que el contraste de la Sección 6: **bajo el diseño experimental de esta entrega, la telemetría no aporta ninguna mejora medible sobre agrupar solo por zona y ventana**. La razón es estructural, no una falla de **c2**: en los tres escenarios ejecutados (Esc-2, Esc-3, Esc-4) cada avería inyectada envía telemetría real al canal PE-U1 exactamente en la misma zona y dentro de la misma ventana de 15 minutos en la que **c1** ya agrupa correctamente por sí solo —la telemetría corrobora una señal que la coincidencia de zona y tiempo ya había capturado, en vez de resolver un caso donde **c1** se equivocaría. El escenario donde la telemetría debería marcar una diferencia real es uno que *no* se ejecutó en esta ronda: dos averías *simultáneas dentro de la misma zona* (ver Sección 15.6), donde **c1** fundiría ambas por coincidir en zona y ventana, mientras que **c2** podría, en principio, usar el patrón de telemetría de cada grupo de equipos para separarlas. Sin ese escenario, la decisión de mantener **c2** en producción frente a quedarse con **c1** —más simple, sin dependencia de **telemetry-service**— no está decidida con dato, aunque el dato de esta ronda ya apunta a que **c1** solo no perdió nada frente a **c2**.

15.5. Control negativo y Escenario 4

El control negativo se cumple: bajo **c0**, Esc-3 (60 abonados) abrió en promedio 60 incidencias —una por ticket, el comportamiento esperado de la línea base—, confirmando que la carga sí reprodujo el fenómeno y que la comparación posterior tiene sentido.

En las 30 corridas de Esc-4 (dos averías simultáneas, en *zonas distintas*, bajo las 3 configuraciones) no se observó ninguna fusión errónea entre las dos averías (intervalo de confianza binomial al 95 %: $[0, 0]$). Esto es el resultado esperado dado que la correlación filtra candidatas por zona antes de aplicar la estrategia (`IncidenciaRepository.findByZoneAndCreatedAtAfter`): una fusión cruzada de zona es estructuralmente imposible en esta implementación, así que Esc-4 tal como está especificado en la guía (zonas distintas) valida el particionado por zona en vez de estresar el caso de fusión más interesante —dos averías simultáneas *dentro de la misma zona*—, que queda como extensión natural del protocolo completo.

15.6. Qué falta para el protocolo completo

Esta corrida demuestra que el mecanismo mide algo real y reproducible, no solo que compila. Para el protocolo completo de la Sección 5.4 de la guía falta: subir a 10 repeticiones por condición, subir la escala de abonados a la especificada (100/500/100+100), y —de forma explícita, no contemplada por el escenario oficial— correr una variante de Esc-4 con dos averías en la *misma* zona, que es el caso donde una fusión errónea sí sería estructuralmente posible y donde el escenario cumpliría su propósito declarado de revelar ese riesgo.

Parte III

Discusión y cierre (síntesis E1–E4)

16. Discusión (Entrega 3)

16.1. Sobre la fragmentación por fecha frente al patrón de acceso real

El rediseño de la fragmentación descrito en la Sección 4 ilustra una tensión frecuente en el diseño de sistemas distribuidos: la clave de fragmentación que exige el requisito formal de la asignatura (`fecha_apertura`) no coincide con el filtro más frecuente del dominio real del problema (`zone`, usado por el control de acceso por rol de técnico y por la mayoría de los reportes operativos). Se priorizó el cumplimiento literal del requisito sobre la preferencia de diseño original, documentando explícitamente en el ADR-0003 el costo aceptado —consultas por zona convertidas en *scatter-gather* sobre las cuatro particiones— en vez de ocultarlo o de forzar un diseño híbrido no solicitado. En un sistema de producción real, esta decisión probablemente se resolvería con una fragmentación compuesta (por ejemplo, subparticiones por zona dentro de cada partición temporal) o con un índice secundario global cubriente sobre `zone`; ambas alternativas quedan fuera del alcance de esta entrega pero se identifican como extensión natural del diseño actual.

16.2. Sobre la colocación cliente–ticket entre microservicios

La limitación más significativa frente a la letra de la guía es la imposibilidad de colocar físicamente `tickets` con la tabla de clientes, porque esta última es propiedad de `auth-service` en una base de datos distinta. Esta tensión —requisito de la guía de Entrega 3 (pensada para un esquema monolítico o de bases de datos compartidas) contra un principio central de la arquitectura de microservicios adoptada desde la Entrega 2 (cada servicio es dueño exclusivo de su propio esquema)— se resolvió documentando la limitación en vez de comprometer el límite de propiedad de datos entre servicios, que es una de las garantías arquitectónicas más valiosas de la descomposición en microservicios [6]: permite evolucionar o escalar `ticket-service` sin coordinar cambios de esquema con `auth-service`. La alternativa de aplicación —resolución de identidad del cliente vía JWT en vez de una clave física compartida— preserva esta garantía a costa de no poder ejecutar un *join* físico dentro del motor de base de datos entre ambas tablas.

16.3. Sobre la instrumentación y la observabilidad

La combinación de pruebas de integración con Testcontainers y métricas Prometheus verificadas con `promtool` (Sección 5) resultó más valiosa de lo previsto inicialmente para diagnosticar el propio incidente de saturación de memoria descrito en esa sección: sin las métricas de reintentos y sin comprender el comportamiento esperado del aislamiento serializable —verificado previamente en las

pruebas de integración—, habría sido más difícil distinguir un problema de recursos del sistema operativo (memoria insuficiente) de un problema de diseño en la capa de datos. Esto refuerza un principio general de sistemas distribuidos: la observabilidad no es solo un requisito de cumplimiento de la rúbrica, sino una herramienta de diagnóstico que se vuelve indispensable precisamente cuando el sistema falla de forma inesperada.

16.4. Sobre la representatividad del pipeline analítico

El pipeline de Spark descrito en la Sección 6 se diseñó deliberadamente para reflejar un caso de uso analítico propio del dominio (reincidencia y *clustering* de incidencias) en vez de aplicar las cinco transformaciones exigidas de forma genérica sobre datos sin relación con el negocio. Esto tiene un costo: los resultados de *speedup* del protocolo experimental (Sección 7) son específicos a un pipeline dominado por una etapa de *join*/ventana con *shuffle* moderado y una etapa final de aprendizaje automático relativamente costosa (vectorización TF-IDF + KMeans), y no necesariamente generalizan a pipelines de Spark dominados por *shuffles* masivos entre tablas de tamaño comparable. Se documenta esta amenaza a la validez externa explícitamente en la Sección 7 en vez de presentar la conclusión del experimento como universalmente aplicable a cualquier carga de trabajo de Spark.

16.5. Alcance no cubierto en esta entrega

Dos elementos mencionados en la guía de la asignatura —un *API Gateway* explícito como punto de entrada único al conjunto de microservicios, y la orquestación completa de los cinco microservicios mediante un único archivo `docker-compose.yml` raíz— no se implementaron en esta entrega y quedan identificados como extensión pendiente: actualmente cada microservicio se levanta y orquesta de forma independiente, sin una capa de enrutamiento y políticas transversales (limitación de tasa, agregación de respuestas) centralizada.

Resuelto en la Entrega 4. Ambos pendientes se cerraron en el ciclo siguiente: `api-gateway` (Spring Cloud Gateway) es hoy el único punto de entrada para los dos clientes, y un único `docker-compose.yml` en la raíz del repositorio orquesta los 17 contenedores del sistema completo, incluyendo un servicio `db-init` de un solo uso que automatiza la inicialización de bases de datos antes manual (Sección 9).

17. Discusión y amenazas a la validez (Entrega 4)

17.1. Interpretación de los resultados

El sistema cumple de forma sólida las características de calidad relacionadas con **estructura y verificabilidad del código** (arquitectura hexagonal con dependencia del dominio hacia adentro, seis patrones GoF con problema real documentado, complejidad ciclomática media de 1.8, capa unitaria de la pirámide de pruebas con 476 pruebas automatizadas ejecutadas en el último ciclo — 315 backend + 128 web + 33 móvil, la suma exacta de la fila «Unitaria» de la Tabla 14 de la Sección 12, donde consta el comando de cada suite — y las relacionadas con **compatibilidad de cliente** (la misma suite E2E pasa en tres motores de navegador distintos, y el código respeta su API mínima declarada). Donde el sistema *no* alcanza el umbral objetivo es, de forma consistente, bajo **conurrencia real**: tanto la eficiencia de desempeño como la fiabilidad se degradan por una limitación de la edición gratuita del motor de base de datos (Sección 14), no por un defecto de diseño de la aplicación. Esta distinción —entre “el diseño no soporta esta carga” y “el entorno de

demostración no soporta esta carga”— es central para interpretar correctamente los resultados: la arquitectura (Raft con quorum de mayoría, *stateless* en cada microservicio, saga por coreografía sin coordinador central) sí está diseñada para escalar horizontalmente; lo que falta es la licencia o el plan de CockroachDB que levante el límite artificial de transacciones concurrentes.

17.2. Alcance y limitaciones

Este proyecto es un sistema de demostración académica, no un despliegue de producción: corre en una sola máquina de desarrollo con Docker Desktop, sobre datos sintéticos, sin balanceo de carga real entre réplicas, y sin el hardware dedicado que un ISP real destinaría a este tipo de sistema. Las conclusiones sobre desempeño y fiabilidad son válidas *para este entorno*, y se documentan explícitamente como tales en vez de generalizarse sin matización a cualquier despliegue posible.

17.3. Amenazas a la validez del proyecto (a nivel general)

Más allá de las amenazas específicas del experimento de carga (ya discutidas en Sección 14), a nivel de todo el proyecto:

Internas:

1. **Un solo desarrollador principal por capa:** cada integrante del equipo se concentró en un área (backend, observabilidad, contratos, documentación), lo que agiliza el desarrollo pero concentra el conocimiento profundo de cada módulo en una sola persona —mitigado parcialmente por la documentación exhaustiva (ADRs, esta guía) que permite a cualquier integrante razonar sobre módulos que no escribió directamente.
2. **Verificación local, no en el entorno de ejecución final:** buena parte de la verificación de este ciclo (pruebas de backend con Testcontainers, cobertura, complejidad ciclomática) se corrió en un contenedor Maven anidado dentro de Docker Desktop en Windows, un entorno con una limitación conocida de redes anidadas (*Docker outside of Docker*) que impidió ejecutar una prueba de integración específica localmente —mitigado porque esa misma prueba sí corre en verde en el *runner* de GitHub Actions, que ejecuta Maven directo sobre la máquina virtual sin anidar Docker.
3. **Datos de demostración generados sintéticamente:** los 504+ tickets de prueba se generaron y rebalancearon con un script SQL propio, no provienen de un ISP real —mitigado documentando explícitamente el script de generación (`resultados/rebalance_demo_tickets.sql`) para que la distribución de estados sea auditable y reproducible.

Externas:

1. **Generalización a otros ISP:** el dominio (zonas de cobertura, categorías de incidencia, políticas de SLA) se modeló según supuestos razonables del equipo, no según un proceso de negocio documentado de un operador real —una limitación externa reconocida desde la Entrega 1.
2. **Generalización a escala de producción:** las cifras de esta evaluación (latencia, tasa de error, cobertura) corresponden a un solo nodo de desarrollo con datos sintéticos; no se puede extrapolar directamente el comportamiento a un despliegue con tráfico real de miles de usuarios concurrentes sin repetir el experimento en ese entorno.

17.4. Reflexión ética

El sistema procesa datos personales reales de abonados de un ISP: ubicación de la incidencia, historial de contacto, y las credenciales de acceso de clientes, técnicos y administradores. El Código de Ética y Conducta Profesional de la ACM [39] establece, entre sus principios generales, evitar daño y respetar la privacidad; frente a esos principios, el equipo identifica lo siguiente:

Ya aplicado en el sistema: las contraseñas nunca se almacenan en texto plano (`BCryptPasswordEncoder`, factor de costo por defecto de Spring Security); los tokens de sesión (JWT) tienen expiración corta y un mecanismo de revocación vía *refresh tokens*; y ningún *endpoint* de administración de la plataforma (*actuator*) expone información sensible del entorno de ejecución (variables, volcado de memoria).

Reconocido como pendiente, no resuelto: la revisión de seguridad de esta entrega (Sección 14) encontró que el sistema no protege el endpoint de inicio de sesión contra intentos repetidos de adivinar una contraseña. Frente a datos reales de personas —técnicos de campo cuya ubicación queda registrada, clientes cuyo domicilio se infiere de su zona de cobertura— esta es una responsabilidad ética concreta, no solo un requisito técnico: el equipo documenta esta brecha de forma explícita, en vez de omitirla, precisamente porque una evaluación honesta de los riesgos reales del sistema es un prerrequisito para poder mitigarlos antes de un eventual uso con usuarios reales.

18. Conclusiones de la Entrega 3 (integración con E4)

Este trabajo abordó dos dimensiones de la Entrega 3 sobre la base de código de microservicios construida en entregas anteriores: la fragmentación horizontal correcta y verificable de la capa de datos, y el procesamiento analítico paralelo de un volumen de datos que excede lo razonable para un solo proceso. Sobre la primera dimensión, se rediseñó la tabla `tickets` de un esquema fragmentado por zona geográfica a uno fragmentado por rango de `fecha_apertura`, verificando formalmente las tres condiciones de corrección de toda fragmentación horizontal —completitud, reconstrucción y disyunción— y documentando explícitamente, en vez de ocultar, los *trade-offs* aceptados: degradación de las consultas por zona a *scatter-gather* sobre cuatro particiones, y la imposibilidad de colocalización física con la tabla de clientes por pertenecer a un microservicio distinto. La corrección del comportamiento transaccional del clúster —en particular, el rechazo explícito de escrituras concurrentes conflictivas propio del aislamiento `SERIALIZABLE`— se verificó empíricamente contra una instancia real de CockroachDB mediante Testcontainers, no mediante inspección manual ni simulación, y se instrumentó con métricas Prometheus validadas sin advertencias por el *linter* oficial de la herramienta.

Sobre la segunda dimensión, se implementó un pipeline de Apache Spark con las cinco categorías de transformación exigidas, aplicadas de forma coherente a un objetivo analítico propio del dominio —reincidencia de clientes y agrupamiento de incidencias por texto y zona— y ejecutado de extremo a extremo sobre un conjunto sintético de 600,000 registros con resultados reproducibles y no triviales: 68.3 % de reincidencia detectada dentro de una ventana de 30 días y una distribución de *clusters* no degenerada. Siguiendo metodología estándar de experimentación en ingeniería de software, se ejecutó además un protocolo formal de 50 corridas contrastando este pipeline contra un baseline secuencial en pandas, con un resultado que no era el esperado y que precisamente por eso es valioso: para este volumen de datos, el baseline secuencial resultó significativamente más rápido que Spark incluso en su configuración de mayor paralelismo ($p \approx 5 \times 10^{-10}$), aunque Spark sí exhibe un *speedup* interno real al escalar de uno a ocho núcleos (ajuste de Amdahl $p \approx 0,53$). Reportar este resultado tal como ocurrió, en vez de solo buscar el resultado que confirmara la hipótesis inicial, es coherente con el principio de honestidad aplicado en todo este documento.

La caracterización de tolerancia a fallos ante la caída de uno y dos nodos, diseñada e implementada en la Sección 5, se ejecutó y midió por completo: el clúster tolera la caída de un nodo sin pérdida de datos ni de disponibilidad de escritura (0 errores en 691 filas, pico de latencia P95 de 1.59s durante la reelección de líder), y pierde disponibilidad de escritura durante 62 segundos al caer dos de tres nodos, recuperándose de inmediato al reincorporar un nodo — ambos resultados

consistentes con el quórum de mayoría exigido por el algoritmo de consenso Raft descrito en la Sección 3.

18.1. Integración con la Entrega 4

Siguiendo el principio de complementariedad E3→E4 de la guía de esta entrega —todo artefacto producido en E3 debe reutilizarse, ampliarse o instrumentarse en E4, no descartarse—, los siguientes artefactos quedan disponibles para consumo directo en el siguiente ciclo:

- **El esquema de base de datos fragmentado** (migración Flyway versionada `V1__init_ticket_schema.sql` desde el Entregable 5 de la guía de cierre, antes un guion suelto sin versionar; ADR-0003) y el propio **clúster de tres nodos de CockroachDB** quedan como la capa de persistencia definitiva del sistema: E4 no necesita rediseñar ni migrar datos, solo construir sobre ella la interfaz web, la interfaz móvil y la observabilidad exigidas en el siguiente ciclo.
- **Las tres métricas Prometheus** instrumentadas en la Sección 5 (`crdb_query_duration_seconds`, `crdb_transaction_retries_total`, `crdb_pool_active_connections`) se emplean tal cual, sin modificación, como fuente de datos del panel de observabilidad consolidado que exige E4 —por eso se instrumentaron en esta entrega y no se aplazaron.
- **Las pruebas de integración con Testcontainers** quedan incorporadas a la suite de pruebas del proyecto como regresión permanente: cualquier cambio futuro sobre `ticket-service` en E4 se sigue verificando contra una instancia real de CockroachDB, no contra una base embebida.
- **El pipeline de Spark y el protocolo experimental** (`spark/src/pipeline.py`, `baseline.py`, `protocol_experiment.py`) quedan como la base analítica reutilizable: E4 puede extenderlos con nuevas transformaciones o exponer sus resultados (reincidencia, *clusters* de incidencias) a través de la interfaz de usuario, sin rehacer la integración JDBC ni el diseño experimental ya validado aquí.

Como trabajo futuro más allá de lo directamente heredable por E4, se identifican dos líneas adicionales: una estrategia de fragmentación compuesta que preserve el beneficio de poda de particiones tanto para consultas temporales como para consultas por zona, y la validación del pipeline analítico sobre un conjunto de datos real de incidencias de un ISP, para contrastar la representatividad del dataset sintético usado en esta entrega frente a la distribución real de texto y de cardinalidad de clientes del dominio.

19. Conclusiones y trabajo futuro

19.1. Cierre del ciclo E1–E4

El proyecto recorrió, a lo largo de cuatro entregas, el ciclo completo de construcción de un sistema distribuido: análisis del dominio y prototipo de comunicación (E1), arquitectura de microservicios sobre Docker Compose (E2), persistencia distribuida real con particionado y tolerancia a fallos verificada experimentalmente, más un pipeline analítico paralelo (E3), y el cierre integral de esta entrega —dos clientes completos, refactor a arquitectura en capas con patrones de diseño reales, una pirámide de pruebas completa, un pipeline de CI/CD de siete *jobs*, observabilidad de las tres señales, y una evaluación experimental de calidad contra ISO/IEC 25010. Ninguna pieza de las entregas anteriores se descartó: el mismo cluster CockroachDB, el mismo esquema de datos particionado, y el mismo pipeline de Spark de la Entrega 3 siguen operando sin cambios detrás de la nueva capa

de clientes y observabilidad. La Tabla 19 hace explícita esta progresión, entrega por entrega, con la evidencia concreta (ADR o sección del documento) que sustenta cada aporte.

Tabla 19: Trazabilidad del ciclo E1–E4: qué aportó cada entrega y dónde queda la evidencia.

Entrega	Aporte arquitectónico	Evidencia
E1	Definición del dominio ISP y primer prototipo monolítico con comunicación por <i>sockets</i> .	Sección 1
E2	Descomposición en cinco microservicios (<code>auth</code> , <code>ticket</code> , <code>ai</code> , <code>notification</code> , <code>report</code>), REST + eventos Kafka, JWT/RBAC.	Sección 1
E3	Fragmentación horizontal sobre CockroachDB de 3 nodos con verificación formal, postura CAP/PACELC mixta, pipeline analítico paralelo con Spark.	ADR-0003, ADR-0004, Secciones 4, 6
E4	Refactor a arquitectura hexagonal con 6 patrones GoF, apps web y móvil nativas, canal de telemetría PE-U1 y correlación CORREL en Incidencias, pipeline de CI/CD de 7 <i>jobs</i> , observabilidad de 3 señales, evaluación experimental ISO/IEC 25010.	ADR-0005, ADR-0006, ADR-0007, ADR-0008, Secciones 9, 10, 11, 12, 13, 14

Nota sobre PE-U1: pese a que su nombre remite a la Entrega 1, se verificó contra el historial completo del repositorio que este canal de telemetría (`sockets` + `gRPC` + reloj de Lamport) nunca se construyó en ninguna entrega anterior — se construyó por primera vez en esta entrega, junto con CORREL (ADR-0007, ADR-0008), y se documenta así explícitamente para no sugerir una continuidad que no existió.

19.2. Aporte de cada integrante

La Tabla 20 declara, por área del repositorio, quién aportó la mayoría de las líneas según el historial de control de versiones. El método es el guion `scripts/aporte_manuscrito.py`, que recorre `git log -numstat` sobre las rutas indicadas, mapea los dieciséis nombres de autor del historial a los seis correos reales que identifican a los cuatro integrantes (algunos usan más de una dirección de *commit*), y descarta las dependencias y los artefactos generados: `node_modules/`, `build/`, `dist/`, `target/`, los archivos de bloqueo de dependencias y los artefactos de compilación de L^AT_EX (`.bbl`, `.blg`, `.aux`, `.toc`, `.out`, `.log`). Esa exclusión importa: hasta la entrega anterior el guion no excluía `.log`, y un `docs/latex/main.log` de 726 líneas que estuvo versionado en un *commit* antiguo inflaba el aporte de C. Pacheco al manuscrito; esta entrega corrige el patrón para que también descarte `.log`, y el método declarado en este epígrafe y el método aplicado en la tabla vuelven a ser el mismo guion. Cualquiera puede reproducir cada fila corriéndolo sobre el *commit* exacto de esta entrega: `python scripts/aporte_manuscrito.py docs/latex/ -commit ceae7375a6433b81361a0403898dd5dc5d66540b`.

Tabla 20: Aporte por área, según líneas reales insertadas en el historial (dependencias y artefactos generados excluidos)

Área	Aporte principal
Backend (<code>services/</code>)	C. Pacheco 31,45 %, J. Álvarez 27,99 %, C. Carpio 21,22 %, R. Cando 19,34 %
Aplicación web (<code>apps/web/</code>)	C. Carpio 83,41 %, J. Álvarez 16,59 %
Aplicación móvil (<code>apps/mobile/</code>)	R. Cando 73,60 %, J. Álvarez 24,63 %, C. Carpio 1,77 %
Manuscrito (<code>docs/latex/</code>)	C. Pacheco 62,95 %, J. Álvarez 15,81 %, R. Cando 12,87 %, C. Carpio 8,37 %
Experimentos (<code>experimentos/</code> , <code>spark/</code>)	R. Cando 73,74 %, C. Pacheco 17,70 %, C. Carpio 5,19 %, J. Álvarez 3,37 %
Observabilidad (<code>ops/</code>)	C. Pacheco 100,00 %
CI/CD (<code>.github/</code>)	C. Carpio 63,02 %, J. Álvarez 25,62 %, C. Pacheco 5,96 %, R. Cando 5,40 %
Arquitectura (<code>docs/adr/</code> , <code>docs/diagrams/</code>)	J. Álvarez 54,83 % , C. Pacheco 18,77 %, R. Cando 13,57 %, C. Carpio 12,83 %
Contratos (<code>pacts/</code> , <code>docs/api/</code>)	J. Álvarez 100,00 %

Nota: cada fila se reproduce con `python scripts/aporte_manuscrito.py < rutas > -commit ceae7375a6433b81361a0403898dd5dc5d66540b`, el commit de cierre de la entrega anterior a esta corrección (el commit de esta propia corrección no se usa como referencia porque medirse a sí mismo es circular). Las líneas de dependencias y de artefactos generados quedan fuera del denominador, como declara el epígrafe.

La distribución es desigual por área, no por persona: cada uno de los cuatro integrantes es autor principal de al menos un área completa del repositorio. Jeremy Álvarez es, en particular, el autor único del contrato de interfaz OpenAPI y de las pruebas de contrato asociadas, y el principal de las tres vistas de arquitectura C4 —trabajo real y sustantivo que, sin embargo, no se tradujo en líneas del manuscrito LaTeX: su aporte ahí es de una sola sección de la Entrega 3 (`conclusiones.tex`, julio) y esta misma tabla, escrita para esta entrega. Declararlo así, en vez de omitirlo, es más útil que una cifra de commits que por sí sola no distingue entre “no trabajó” y “trabajó en código y contratos, no en prosa”.

19.3. Contribuciones concretas de esta entrega

- Refactor verificable de `ticket-service` a cuatro capas con seis patrones GoF, cada uno documentado con el problema real que resolvía antes del cambio; el mismo patrón de capas se extendió a `auth-service` y `report-service` (Sección 9.4), con alcance declarado explícitamente para los tres servicios que quedaron fuera.
- Dos aplicaciones cliente completas y funcionales (web y móvil) consumiendo la misma API a través de un único *gateway*.
- Contratos consumidor-proveedor verificados en verde contra el sistema real, no contra *mocks* estáticos.
- Un pipeline de CI/CD de diez *jobs* que actúa como compuerta de calidad real.
- Observabilidad de extremo a extremo con las tres señales correlacionadas por un identificador común.
- Una evaluación experimental honesta contra ISO/IEC 25010, que reporta tanto lo que cumple el umbral objetivo como lo que no, con causa raíz identificada en cada caso.

- Cobertura de pruebas de **ticket-service** en 96.6 % de líneas propias (por encima del objetivo del 70 %), priorizando con el propio reporte de JaCoCo las clases reales con mayor número de líneas sin cubrir.

19.4. Trabajo futuro

Directamente derivado de las brechas reales encontradas en esta misma entrega (Secciones 12, 14 y 17):

1. Obtener una licencia de CockroachDB (o migrar a CockroachDB Serverless) para levantar el límite de transacciones concurrentes que hoy degrada la eficiencia y la fiabilidad bajo carga real.
2. Agregar protección contra fuerza bruta (*rate-limiting* o bloqueo temporal de cuenta) en el inicio de sesión de **auth-service**.
3. Ejecutar el protocolo de evaluación ISO 25010 a escala completa (10 repeticiones, 5 y 10 minutos por escenario) una vez que el cronograma del equipo lo permita, siguiendo exactamente la metodología ya validada a escala reducida en esta entrega.
4. Completar las pruebas instrumentadas de Android en un dispositivo/emulador real, y ampliar su cobertura más allá del flujo de inicio de sesión.

Referencias

- [1] International Organization for Standardization, “ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model.” ISO Standard, 2023.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [3] OpenTelemetry Authors, “OpenTelemetry specification, version 1.32.” CNCF Sandbox Project Documentation, 2024.
- [4] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” Tech. Rep. RFC 7519, IETF, 2015.
- [5] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2nd ed., 2021.
- [7] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference (AFIPS ’67)*, pp. 483–485, ACM, 1967.
- [8] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pp. 305–319, USENIX Association, 2014.
- [9] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, “CockroachDB: The resilient geo-distributed SQL database,” in *Proceedings of the*

- 2020 ACM SIGMOD International Conference on Management of Data, pp. 1493–1509, ACM, 2020.
- [10] E. Brewer, “CAP twelve years later: How the rules have changed,” *IEEE Computer*, vol. 45, no. 2, pp. 23–29, 2012.
 - [11] D. Abadi, “Consistency tradeoffs in modern distributed database system design: CAP is only part of the story,” *IEEE Computer*, vol. 45, no. 2, pp. 37–42, 2012.
 - [12] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*. Springer, 3rd ed., 2011.
 - [13] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pp. 15–28, USENIX Association, 2012.
 - [14] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, “Big data analytics on apache spark,” *International Journal of Data Science and Analytics*, vol. 1, no. 3–4, pp. 145–164, 2016.
 - [15] B. Chambers and M. Zaharia, *Spark: The Definitive Guide: Big Data Processing Made Simple*. O’Reilly Media, 2018.
 - [16] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC ’10)*, pp. 143–154, ACM, 2010.
 - [17] M. Waseem, P. Liang, M. Shahin, A. Di Salle, and G. Márquez, “Design, monitoring, and testing of microservices systems: The practitioners’ perspective,” *Journal of Systems and Software*, vol. 182, p. 111061, 2021.
 - [18] M. Hui, L. Wang, H. Li, R. Yang, Y. Song, H. Zhuang, D. Cui, and Q. Li, “Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,” *Journal of Systems and Software*, vol. 220, p. 112232, 2025.
 - [19] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
 - [20] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, 1991.
 - [21] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
 - [22] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
 - [23] J. L. Gustafson, “Reevaluating amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
 - [24] Cockroach Labs, “CockroachDB documentation.” <https://www.cockroachlabs.com/docs/>, 2024. Accedido: julio 2026.

- [25] A. Vargha and H. D. Delaney, “A critique and improvement of the CL common language effect size statistics of McGraw and Wong,” *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.
- [26] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2017.
- [27] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.
- [28] A. I. Wasserman, “Software engineering issues for mobile application development,” in *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research (FoSER)*, pp. 397–400, ACM, 2010.
- [29] W. S. El-Kassas, B. A. Abdullah, A. H. Yousef, and A. M. Wahba, “Taxonomy of cross-platform mobile applications development approaches,” *Ain Shams Engineering Journal*, vol. 8, no. 2, pp. 163–190, 2017.
- [30] A. Biørn-Hansen, T.-M. Grønli, G. Ghinea, and S. Alouneh, “An empirical study of cross-platform mobile development in industry,” *Wireless Communications and Mobile Computing*, p. 5743892, 2019.
- [31] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.
- [32] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook*. Portland, OR, USA: IT Revolution Press, 2nd ed., 2021.
- [33] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, “Dapper, a large-scale distributed systems tracing infrastructure,” Tech. Rep. dapper-2010-1, Google, Inc., 2010.
- [34] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O’Reilly Media, 2016.
- [35] S. Brown, “The C4 model for visualising software architecture.” <https://c4model.com/>, 2018. Accedido: septiembre 2026.
- [36] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [37] P. Ammann and J. Offutt, *Introduction to Software Testing*. Cambridge, UK: Cambridge University Press, 2nd ed., 2016.
- [38] OWASP Foundation, “OWASP top 10 web application security risks — 2021.” Online report, 2021.
- [39] ACM Council, “ACM code of ethics and professional conduct.” Online policy document, 2018.